



DEPARTMENT OF COMPUTER ENGINEERING
HWSW CODESIGN
MMH: HSCD446164

Group: 08

Name:

Trần Ngọc Dũng - 23119134

Huỳnh Chí Thiện - 23119207

Trần Chí Nguyên - 23119179

Nguyễn Hữu Trí - 22119244

Nguyễn Viết Nam – 23119173

Giới thiệu

Bài Lab tập trung vào thiết kế hệ thống Zynq SoC sử dụng hardware interrupts bằng Vivado IP Integrator và SDK/Vitis. Người học sẽ xây dựng hệ thống gồm AXI GPIO, AXI Timer và ARM Cortex-A9 để xử lý interrupt từ Push Buttons và Timer trên Zynq FPGA.

Mục tiêu bài Lab

- + Hiểu cơ chế interrupt trong Zynq SoC
- + Thiết kế hệ thống interrupt-based trên FPGA SoC
- + Sử dụng AXI GPIO và AXI Timer
- + Cấu hình Fabric Interrupt giữa PL và PS
- + Viết chương trình xử lý interrupt trên ARM Cortex-A9
- + Kết nối nhiều nguồn interrupt bằng Concat IP
- + Điều khiển LEDs bằng interrupt-driven software

Công cụ cần thiết

Phần cứng

- + ZedBoard hoặc PynQ-Z2
- + USB JTAG/UART

Phần mềm

- + Vivado Design Suite 2019.2
- + Vivado SDK / Vitis
- + Vivado Hardware Server

Kết quả đạt được

- + Xây dựng hệ thống Zynq sử dụng interrupt
- + Xử lý interrupt từ Push Buttons và Timer
- + Kết nối nhiều nguồn interrupt trong Zynq
- + Generate và nạp bitstream xuống FPGA
- + Chạy ứng dụng interrupt-driven trên ARM Cortex-A9
- + Hiểu nguyên lý interrupt trong FPGA SoC nhúng

Phần công việc thực hiện

A. Câu hỏi

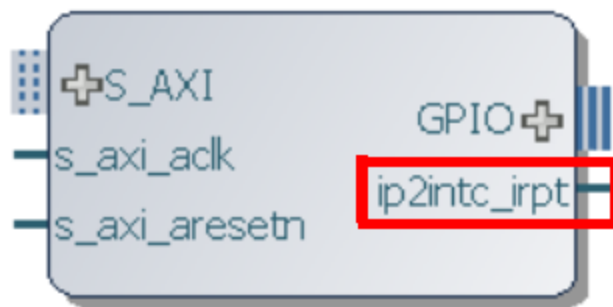
1. Giải thích cơ chế interrupt trong hệ thống Zynq SoC. Interrupt giúp cải thiện quá trình xử lý so với phương pháp polling như thế nào?

- Cơ chế Interrupt trong hệ thống Zynq SoC

Cơ chế ngắt (interrupt) trong Zynq SoC là một quá trình giao tiếp giữa phần logic lập trình được (Programmable Logic - PL) và hệ thống xử lý (Processing System - PS) để thông báo về các sự kiện phần cứng

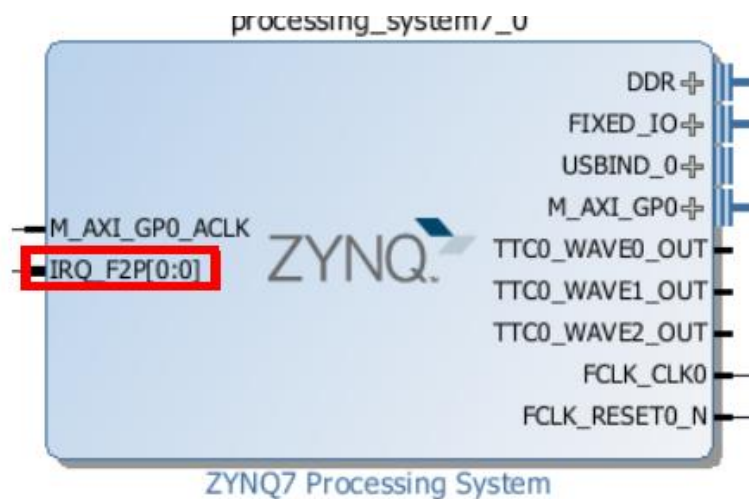
Quy trình hoạt động bao gồm các thành phần chính:

- + Nguồn phát ngắt (PL): Các khối IP trong PL như AXI GPIO (kết nối với nút nhấn) hoặc AXI Timer được cấu hình để tạo ra tín hiệu ngắt. Ví dụ, khối AXI GPIO phải được bật tùy chọn Enable Interrupt để có thể phát tín hiệu ip2intc_irqt khi trạng thái nút nhấn thay đổi.



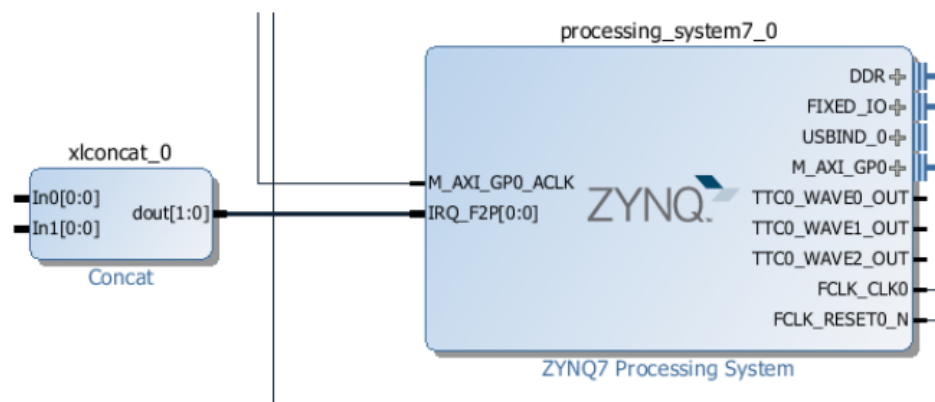
Hình: Khối GPIO với cổng ngắt

- + Cổng nhận ngắt (PS): Để PS có thể nhận được tín hiệu từ PL, người thiết kế cần bật tính năng Fabric Interrupts và cổng IRQ_F2P (Interrupt Request từ Fabric đến PS) trong cấu hình của ZYNQ7 Processing System.

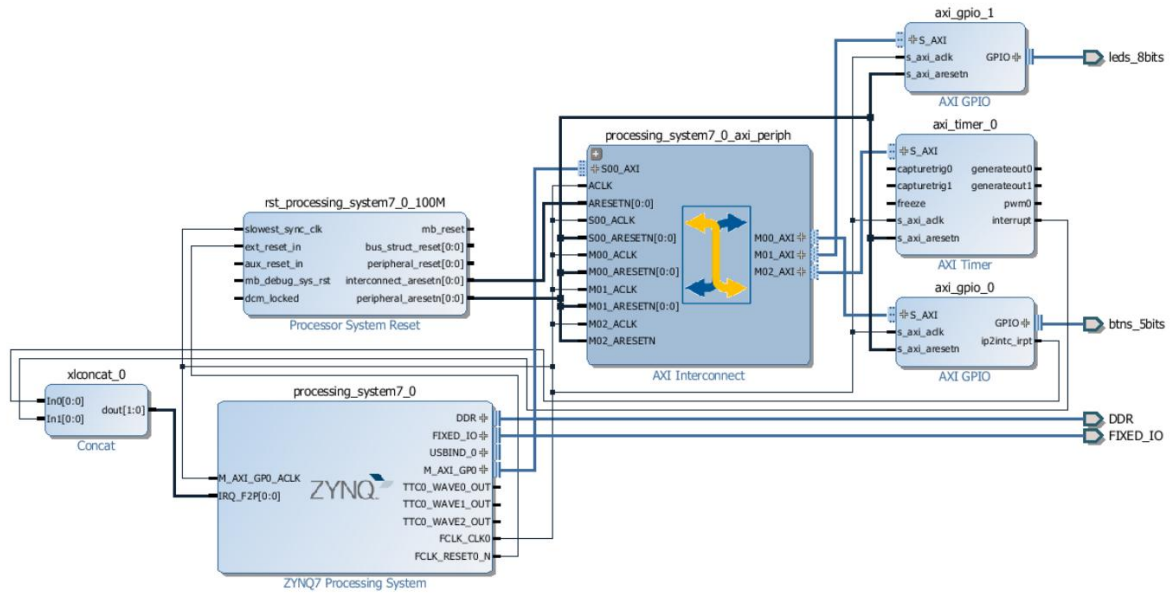


Hình: Zynq PS với cổng ngắt

- + Quản lý nhiều nguồn ngắt: Trong trường hợp hệ thống có nhiều nguồn ngắt đồng thời (như từ cả GPIO và Timer), khối Concat IP được sử dụng để gộp các đường ngắt này lại trước khi đưa vào cổng IRQ_F2P của PS.

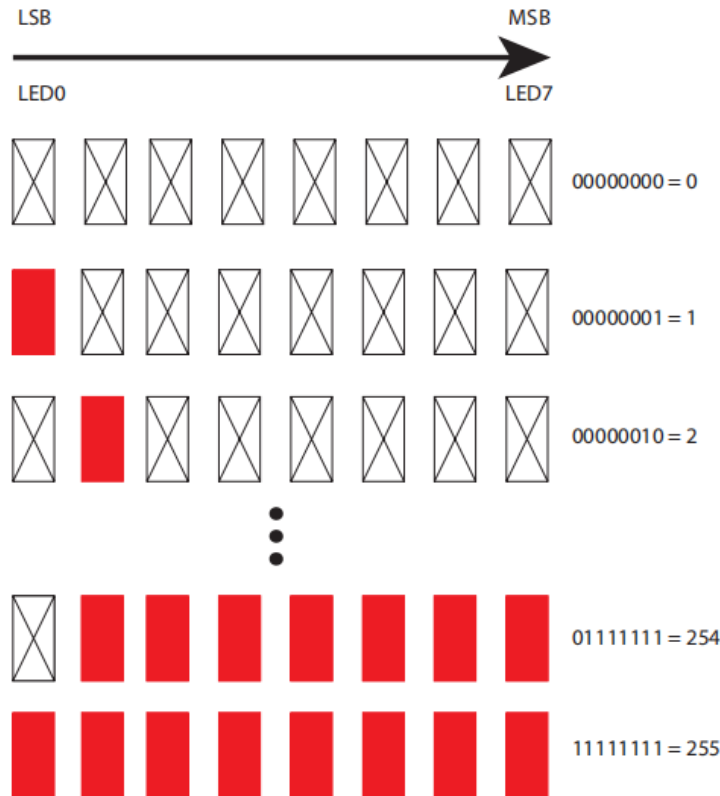


Hình: Khối Concat trong thiết kế sơ đồ khối



Hình: Hệ thống hoàn chỉnh với nhiều nguồn ngắt (bao gồm GPIO và Timer)

- + Xử lý phần mềm: Khi tín hiệu ngắt truyền đến, bộ vi xử lý ARM Cortex-A9 sẽ tạm dừng chương trình hiện tại để thực thi một chương trình xử lý ngắt chuyên dụng (interrupt-driven software) nhằm thực hiện các tác vụ cụ thể, chẳng hạn như điều khiển đèn LED.

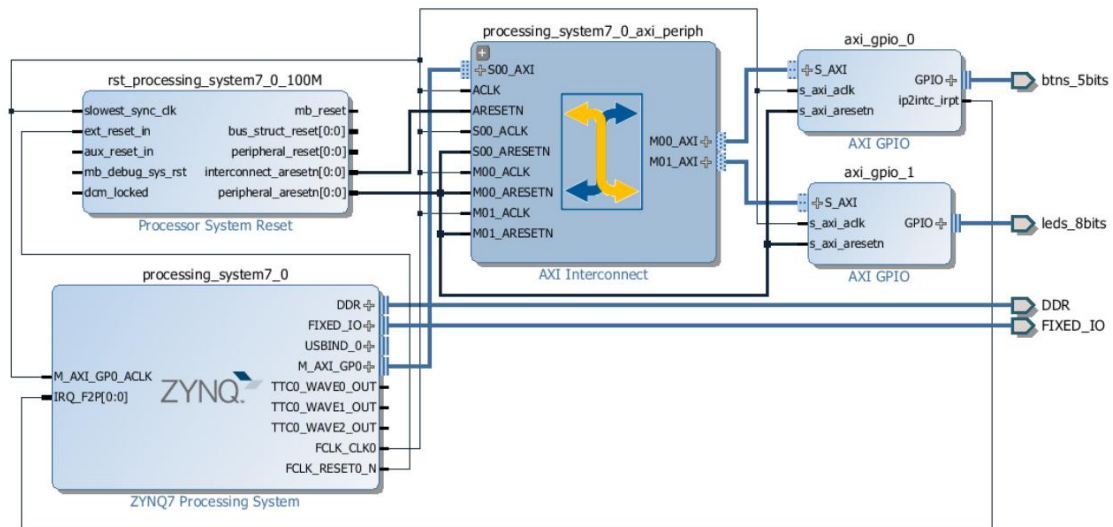


Hình: Các trạng thái nhấp nháy của đèn LED (hiển thị giá trị đếm nhị phân)

- Ưu điểm của Interrupt so với phương pháp Polling

Phương pháp ngắt giúp cải thiện hiệu suất hệ thống đáng kể so với phương pháp Polling (thăm dò) nhờ các yếu tố sau:

- + Tối ưu hóa tài nguyên CPU: Trong phương pháp Polling, CPU phải liên tục chạy vòng lặp để kiểm tra trạng thái của thiết bị (ví dụ: kiểm tra xem nút nhấn có được nhấn hay không). Với Interrupt, CPU chỉ xử lý khi có sự kiện thực sự xảy ra, giúp tiết kiệm tài nguyên để thực hiện các tác vụ quan trọng khác.
- + Phản hồi tức thì: Hệ thống dựa trên cơ chế ngắt (interrupt-based system) cho phép phản hồi ngay lập tức các sự kiện từ ngoại vi (nút nhấn, bộ định thời) mà không cần chờ đợi đến lượt kiểm tra trong vòng lặp phần mềm.
- + Nâng cao hiệu quả xử lý đa nhiệm: Cơ chế này cho phép hệ thống quản lý đồng thời nhiều nguồn sự kiện khác nhau một cách hiệu quả thông qua việc kết nối các nguồn ngắt từ PL về PS, giúp việc điều khiển các thiết bị như LED và Push Buttons trở nên linh hoạt và chính xác hơn.



Hình: Hệ thống vi xử lý Zynq với các ngắt (sơ đồ kết nối hoàn chỉnh cho một nguồn ngắt)

2.Vai trò của AXI GPIO trong bài là gì? Vì sao AXI GPIO có thể tạo interrupt từ Push Buttons?

Hệ thống sử dụng hai khối GPIO AXI với chức năng phân biệt rõ ràng:

axi_gpio_0 → kết nối Push Buttons (btns_5bits / btns_4bits) └─ Nguồn tạo interrupt
axi_gpio_1 → kết nối LEDs (leds_8bits / leds_4bits) └─ Hiển thị giá trị counter

Một khối AXI GPIO (axi_gpio_1) kết nối trực tiếp với các đèn LED trên board phát triển để điều khiển trạng thái hiển thị của chúng. Khối AXI GPIO còn lại (axi_gpio_0) kết nối với các nút nhấn (Push Buttons). Vai trò cốt lõi của khối này là giám sát trạng thái phần cứng của nút bấm, chuyển đổi các tín hiệu vật lý từ nút bấm thành dữ liệu số và tạo ra tín hiệu yêu cầu ngắt (Interrupt Request) gửi tới lõi xử lý ARM Cortex-A9 thuộc vùng xử lý trung tâm (Processing System - PS) mỗi khi có nút được nhấn.

Cả hai khối này đều giao tiếp với bộ vi xử lý Zynq thông qua chuẩn bus AXI (Advanced eXtensible Interface) cho phép phần mềm (ứng dụng C chạy trên PS) đọc dữ liệu từ nút bấm hoặc ghi dữ liệu điều khiển đèn LED.

Khối IP AXI GPIO của Xilinx được thiết kế tích hợp sẵn mạch phần cứng quản lý ngắt (Interrupt Controller nội bộ). Trong cấu hình của Vivado ẻ AXI GPIO được tạo ngắt cần thực hiện hai bước cấu hình quan trọng:

Bước 1 — Kích hoạt tính năng ngắt trong Cấu hình IP:

Bước tiếp theo là nhấp đúp chuột để mở axi_gpio_0tab **Cấu hình IP** và mở phần “**Bật ngắt**”. Đây là nơi khối GPIO hoạt động ip2intc_irpt.

Trước khi bật: [axi_gpio_0] → chỉ có GPIO port

Sau khi bật: [axi_gpio_0] — GPIO port + ip2intc_irpt port

Bước 2 — Bên trong phần cứng cơ chế trong AXI GPIO:

AXI GPIO có bộ ghi nội bộ theo dõi việc thay đổi trạng thái của các đầu vào chân. Khi thay đổi (ấn hoặc thêm) trạng thái Push Button, GPIO IP detect edge transition và kích hoạt tín hiệu ngắt `ip2intc_irqpt = HIGH`, cảnh báo cho quá trình xử lý PS.

3. AXI Timer được sử dụng để làm gì trong hệ thống interrupt-based? Trình bày cách Timer tạo interrupt cho ARM Cortex-A9.

1. Mục đích của AXI Timer

Trong một hệ thống dựa trên ngắt (interrupt-based system), AXI Timer được sử dụng để định kỳ tạo ra các tín hiệu ngắt (interrupt requests) gửi đến bộ xử lý (như ARM Cortex-A9). Nó đếm thời gian (đếm lên hoặc đếm xuống) dựa trên xung nhịp hệ thống, và khi đạt đến một giá trị giới hạn hoặc đếm về 0, Timer sẽ hết hạn (expire) và tự động kích hoạt một cờ ngắt. Bộ vi xử lý có thể dùng sự kiện này để thực hiện các tác vụ định kỳ, đo lường thời gian, hoặc cập nhật bộ đếm mà không cần phải liên tục kiểm tra (polling).

2. Cách Timer tạo interrupt cho ARM Cortex-A9

Quá trình AXI Timer tạo ngắt cho ARM Cortex-A9 trong hệ thống Zynq diễn ra theo các bước sau:

Bước 1: Khởi tạo và cấu hình Timer: AXI Timer được nạp một giá trị khởi tạo (ví dụ:

`TMR_LOAD`) và được phần mềm cấu hình để đếm (chẳng hạn đếm lùi về 0) thông qua bus AXI.

Bước 2: Phát tín hiệu ngắt: Khi Timer đếm hết giá trị và bị tràn (overflow) hoặc về 0, nó sẽ thay đổi trạng thái chân tín hiệu ngắt (interrupt request pin).

Bước 3: Kết nối vật lý trên phần cứng: Chân tín hiệu ngắt của AXI Timer được nối với ngõ vào ngắt của khối ZYNQ7 Processing System (PS). Nếu có nhiều nguồn ngắt (ví dụ GPIO và Timer), chúng sẽ được đưa qua khối Concat để gom lại thành một bus tín hiệu ngắt trước khi kết nối vào PS.

Bước 4: Xử lý tại bộ điều khiển ngắt (GIC): Tín hiệu từ AXI Timer truyền từ PL (Programmable Logic) sang PS thông qua cổng Interrupt (`IRQ_F2P`). Bộ Generic Interrupt Controller (GIC) bên trong PS nhận tín hiệu, ưu tiên nó và thông báo cho nhân ARM Cortex-A9.



Hình : Cấu hình Zynq PS để cho phép nhận tín hiệu ngắt từ PL

Bước 5: Thực thi Interrupt Handler: ARM Cortex-A9 dừng công việc hiện tại, lưu ngữ cảnh, và nhảy tới hàm xử lý ngắt (Interrupt Service Routine - ISR) tương ứng của Timer (đã được phần mềm đăng ký trước đó). Sau khi xử lý xong (ví dụ: tăng biến đếm, xóa cờ ngắt của Timer), CPU tiếp tục chạy chương trình chính.

4.Fabric Interrupt giữa Programmable Logic (PL) và Processing System (PS) có vai trò gì? Vì sao cần bật cổng IRQ_F2P trong ZYNQ7 Processing System?

Vai trò của Fabric Interrupt:

Fabric Interrupt đóng vai trò là "con đường truyền tin" (cổng giao tiếp) để các thành phần phần cứng nằm trong vùng Programmable Logic (PL - FPGA) gửi tín hiệu yêu cầu ngắt (Interrupt Request - IRQ) đến bộ xử lý Processing System (PS - ARM Cortex-A9).

Trong hệ thống Zynq, CPU chỉ có thể biết được các sự kiện xảy ra bên trong PL (như khi nút nhấn được nhấn hoặc bộ đếm thời gian tràn) nếu PL gửi một tín hiệu thông báo thông qua các đường Fabric Interrupt này.

Vì sao cần bật cổng IRQ_F2P trong ZYNQ7 Processing System?

IRQ_F2P là viết tắt của Interrupt Request from FPGA to PS. Việc bật cổng này trong cấu hình của ZYNQ7 Processing System là bắt buộc vì các lý do sau:

- Kích hoạt bộ điều khiển ngắt (GIC): Mặc định, các cổng kết nối giữa PL và PS bị vô hiệu hóa để tối ưu hóa tài nguyên. Bằng cách bật IRQ_F2P trong cấu hình (tại mục Interrupts trong PS-

Configuration), bạn đang ra lệnh cho bộ điều khiển ngắt chung (Generic Interrupt Controller - GIC) bên trong PS mở "cánh cửa" để chấp nhận các tín hiệu ngắt đến từ phía PL.

- Kết nối vật lý: Nếu không bật cổng này, trong giao diện Vivado IP Integrator, cổng IRQ_F2P sẽ không xuất hiện trên khối ZYNQ7 PS. Khi đó, bạn sẽ không có nơi nào để nối các tín hiệu ngắt từ các IP khác (như AXI GPIO, AXI Timer) dẫn về CPU.
- Cơ chế phân cấp: Khi cổng này được kích hoạt, nó cho phép các ngắt từ PL được đưa vào GIC của PS. Sau đó, phần mềm (thông qua mã nguồn C) có thể cấu hình để nhận diện, ưu tiên và xử lý các ngắt này theo thời gian thực.

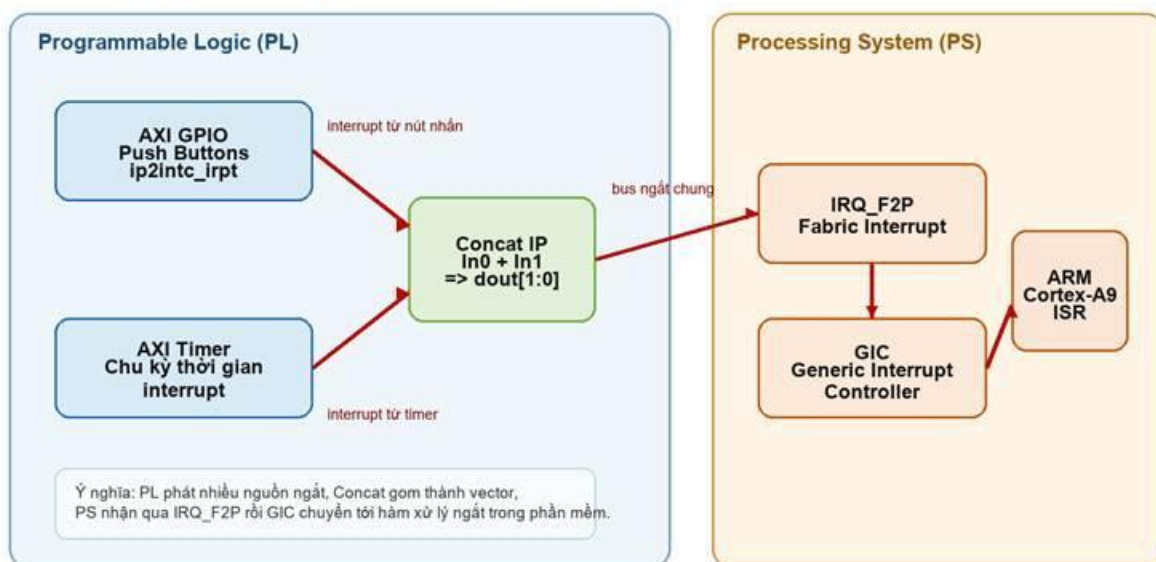
5. Concat IP được dùng để làm gì khi hệ thống có nhiều nguồn interrupt? Trình bày quy trình kết nối interrupt từ AXI GPIO và AXI Timer về Processing System.

I. TỔNG QUAN

Trong hệ thống Zynq SoC, Processing System (PS) sử dụng bộ xử lý ARM Cortex-A9 để chạy chương trình phần mềm, còn các khối IP như AXI GPIO và AXI Timer được triển khai trong Programmable Logic (PL). Khi các IP trong PL cần báo sự kiện cho bộ xử lý, chúng sẽ phát tín hiệu interrupt về PS để chương trình C xử lý bằng interrupt handler.

Ở bài này, hệ thống có nhiều nguồn interrupt: AXI GPIO tạo interrupt khi có sự kiện từ Push Buttons, còn AXI Timer tạo interrupt theo chu kỳ thời gian hoặc khi bộ đếm đạt giá trị cấu hình. Vì có nhiều nguồn ngắt cùng cần đưa về Processing System, ta sử dụng Concat IP để gom các tín hiệu interrupt rời rạc thành một bus interrupt chung.

Sơ đồ tổng quan đường đi của interrupt trong Zynq SoC



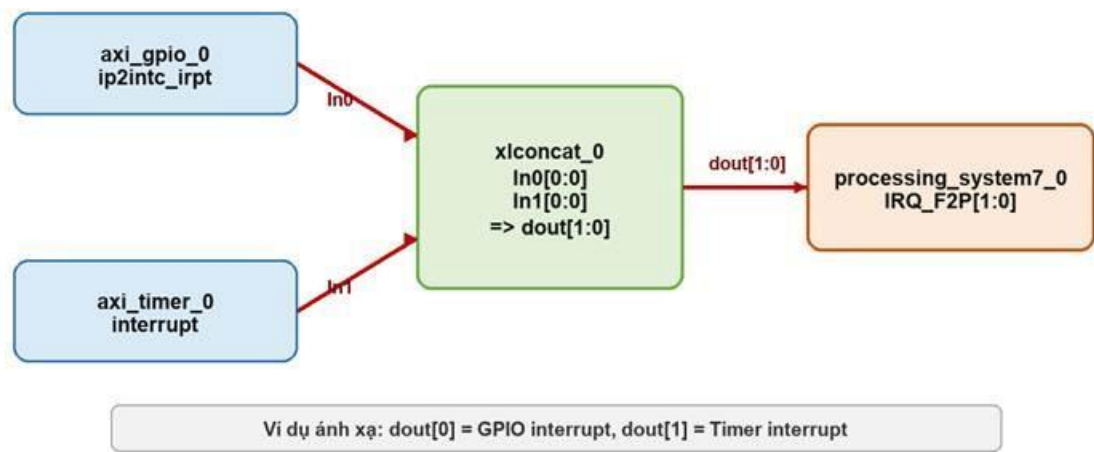
Hình 1: Sơ đồ tổng quan đường đi của interrupt từ PL về PS

II. VAI TRÒ CỦA CONCAT IP

Concat IP là khối dùng để ghép nhiều tín hiệu đơn lẻ thành một vector nhiều bit. Trong hệ thống interrupt của bài, Concat IP nhận interrupt từ AXI GPIO và AXI Timer ở các ngõ vào riêng, sau đó đưa ngõ ra dout[1:0] về cổng IRQ_F2P của ZYNQ7 Processing System.

Nhờ Concat IP, nhiều nguồn interrupt trong PL có thể đi chung qua một đường bus về PS. Khi interrupt xảy ra, phần mềm trên ARM Cortex-A9 sẽ dựa vào bộ điều khiển ngắt và driver để xác định nguồn ngắt đến từ AXI GPIO hay AXI Timer.

Concat IP ghép nhiều nguồn interrupt thành một vector



Hình 2: Sơ đồ ghép interrupt từ AXI GPIO và AXI Timer bằng Concat IP

Nguồn interrupt	Tín hiệu	Vai trò
AXI GPIO	ip2intc_irpt	Phát ngắt khi có sự kiện từ Push Buttons.
AXI Timer	interrupt	Phát ngắt khi Timer đạt chu kỳ hoặc tràn bộ đếm.
Concat IP	dout[1:0]	Ghép các interrupt thành bus đưa về IRQ_F2P của PS.

III. QUY TRÌNH KẾT NỐI INTERRUPT

Bước 1: Bật interrupt cho các IP trong PL

Trước hết, cần cấu hình các IP có khả năng tạo interrupt. Đối với AXI GPIO, bật chức năng interrupt để khối GPIO phát tín hiệu ip2intc_irpt khi có sự kiện từ Push Buttons. Đối với AXI Timer, sử dụng cổng interrupt để báo cho PS khi Timer tạo sự kiện ngắt.

Bước 2: Thêm Concat IP vào Block Design

Trong Vivado IP Integrator, thêm khối Concat IP vào Block Design. Vì bài Lab có hai nguồn interrupt chính là AXI GPIO và AXI Timer, Concat được cấu hình với hai ngõ vào In0 và In1. Mỗi ngõ vào nhận một tín hiệu interrupt riêng từ các IP trong PL.

Bước 3: Nối interrupt từ AXI GPIO và AXI Timer vào Concat

Thực hiện kết nối axi_gpio_0/ip2intc_irpt vào xlconcat_0/In0 và axi_timer_0/interrupt vào xlconcat_0/In1. Sau bước này, ngõ ra của Concat sẽ là bus dout[1:0], đại diện cho hai nguồn interrupt khác nhau.

- In0 nhận interrupt từ AXI GPIO, dùng cho sự kiện Push Buttons.
- In1 nhận interrupt từ AXI Timer, dùng cho sự kiện thời gian.
- dout[1:0] là bus interrupt tổng hợp đưa về Processing System.

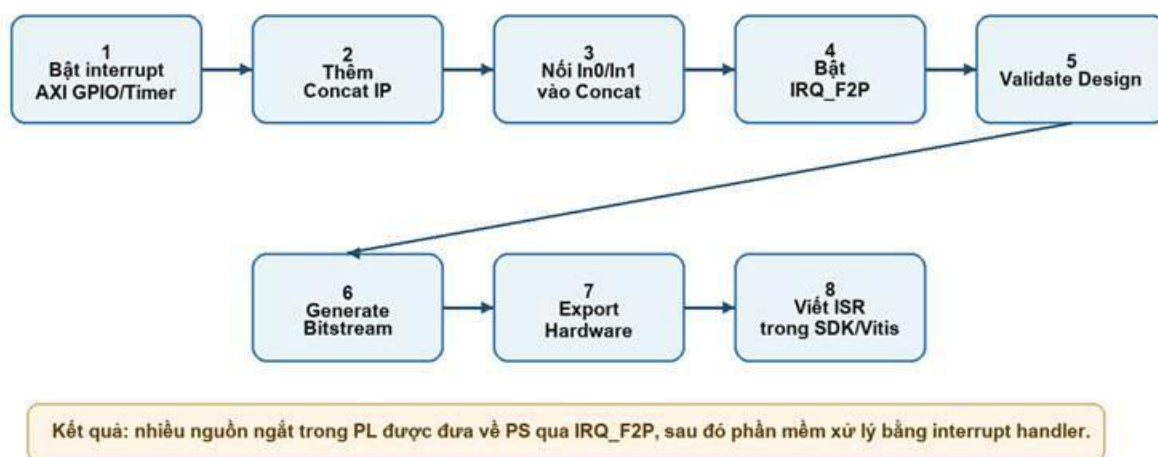
Bước 4: Bật Fabric Interrupt trên ZYNQ7 Processing System

Để Processing System nhận được interrupt từ PL, mở cấu hình ZYNQ7 Processing System và bật Fabric Interrupts. Sau khi bật, khối PS xuất hiện cổng IRQ_F2P. Đây là cổng nhận các tín hiệu ngắt từ Fabric/PL đưa về bộ xử lý ARM.

Bước 5: Kiểm tra sơ đồ Block Design hoàn chỉnh

Sau khi hoàn tất kết nối, Block Design gồm ZYNQ7 Processing System, AXI Interconnect, AXI GPIO, AXI Timer, Concat IP và Processor System Reset. AXI GPIO được nối với Push Buttons và LEDs, AXI Timer được nối vào hệ thống AXI, còn các tín hiệu interrupt được gom qua Concat IP rồi đưa về IRQ_F2P của Processing System.

Lưu đồ quy trình kết nối interrupt trong Vivado



Hình 3: Lưu đồ quy trình kết nối interrupt và triển khai hệ thống

IV. KẾT LUẬN

Concat IP đóng vai trò quan trọng trong hệ thống có nhiều nguồn interrupt. Thay vì nối riêng từng interrupt trực tiếp về PS, Concat gom các tín hiệu ngắt từ AXI GPIO và AXI Timer thành một bus chung rồi đưa vào cổng IRQ_F2P của ZYNQ7 Processing System. Nhờ đó, chương trình C chạy trên ARM Cortex-A9 có thể xử lý nhiều sự kiện phần cứng theo cơ chế interrupt-driven, giúp hệ thống phản hồi nhanh hơn và hiệu quả hơn so với phương pháp polling.

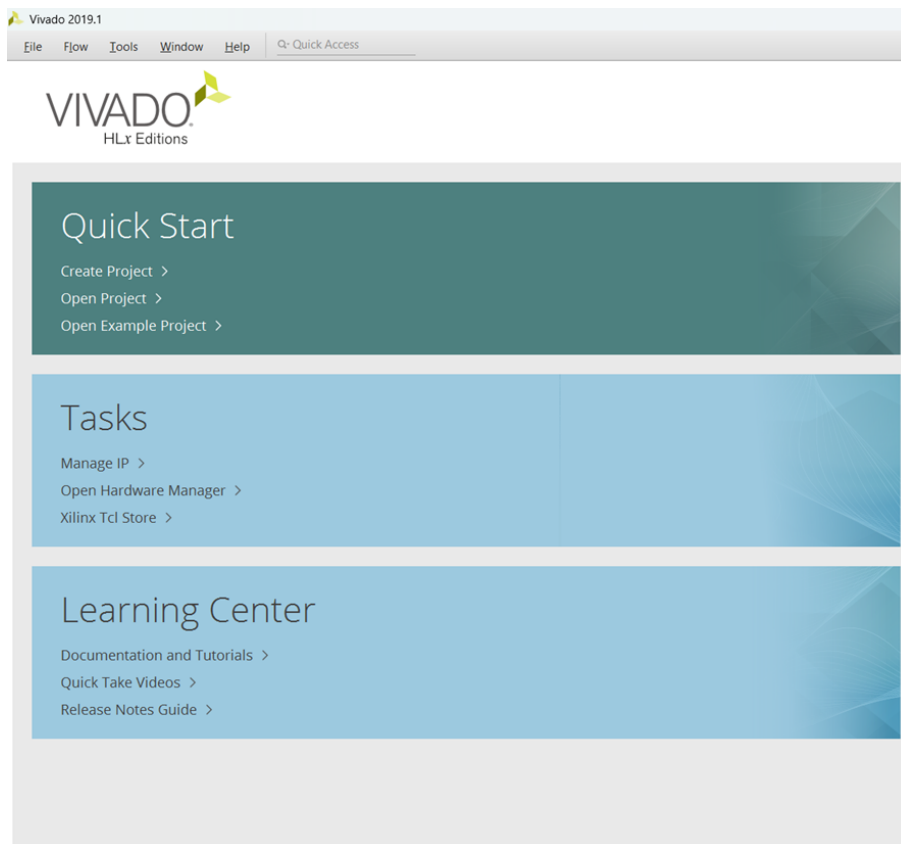
B. Thực Hành

2A. Tạo Project Vivado

Phần này trình bày các bước tạo project ban đầu trong Vivado, chọn đúng board phát triển và xác nhận cấu hình trước khi chuyển sang thiết kế bằng IP Integrator.

Bước 1: Khởi động Vivado

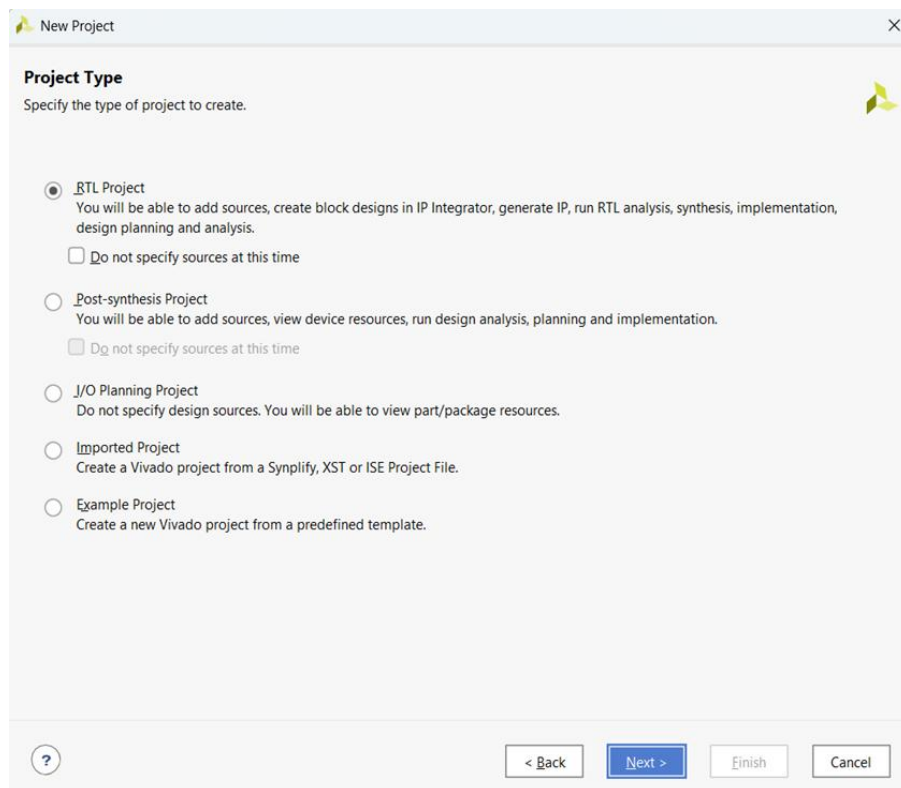
Mở Vivado Design Suite. Tại giao diện Quick Start, người dùng có thể tạo project mới, mở project đã có hoặc truy cập các tài nguyên hướng dẫn của Xilinx. Đây là điểm bắt đầu để xây dựng hệ thống Zynq SoC trong bài Lab.



Hình 1: Giao diện khởi động Vivado - Quick Start

Bước 2: Khởi tạo Project mới

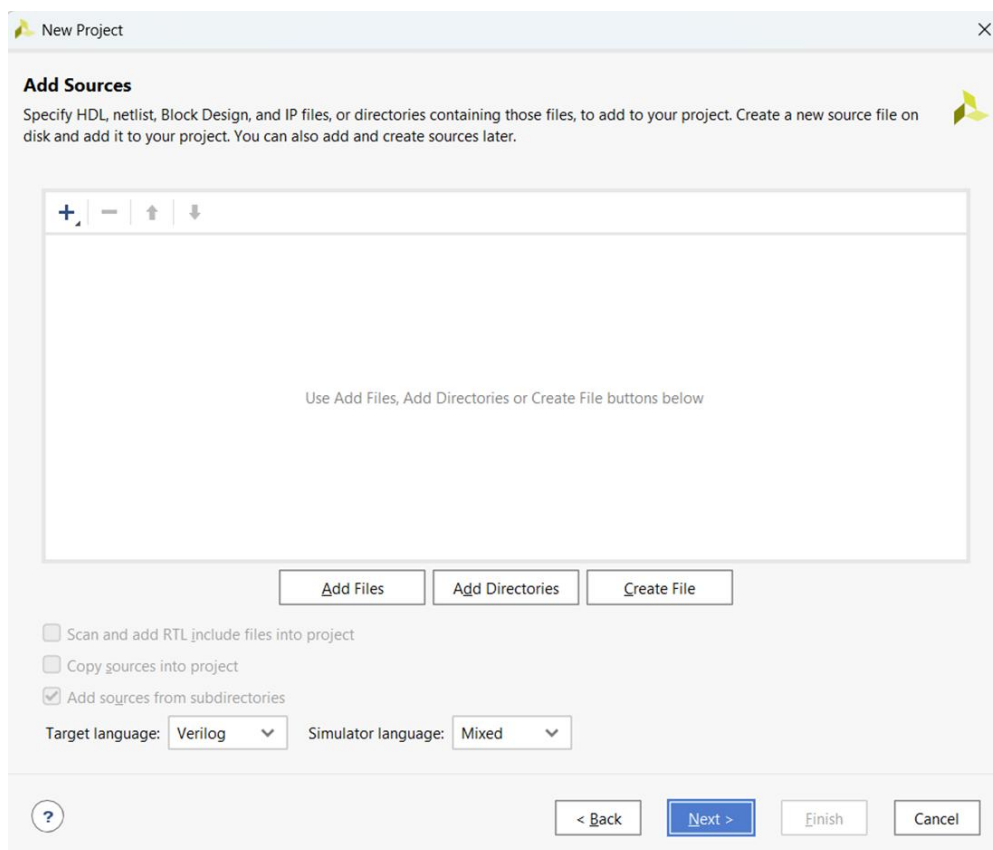
Chọn Create Project để mở New Project Wizard. Ở bước này, Vivado hướng dẫn người dùng tạo project theo từng màn hình cấu hình, từ tên project, thư mục lưu trữ đến kiểu thiết kế.



Hình 2: Hộp thoại New Project Wizard - chọn kiểu Project

Bước 3: Bỏ qua bước thêm source HDL ban đầu

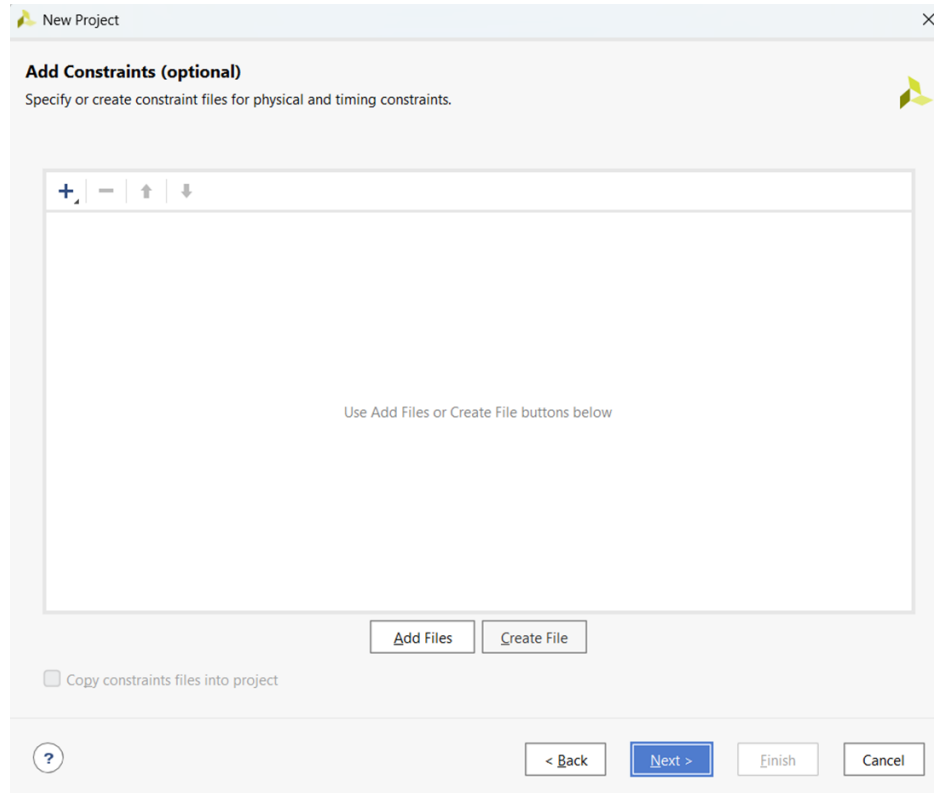
Ở màn hình Add Sources, chưa cần thêm file HDL vì phần cứng của bài Lab sẽ được xây dựng chủ yếu bằng IP Integrator và Block Design. Có thể để trống bước này và bỏ sung source sau nếu cần.



Hình 3: Add Sources - bỏ qua bước thêm source HDL ban đầu

Bước 4: Bỏ qua bước thêm file Constraints ban đầu

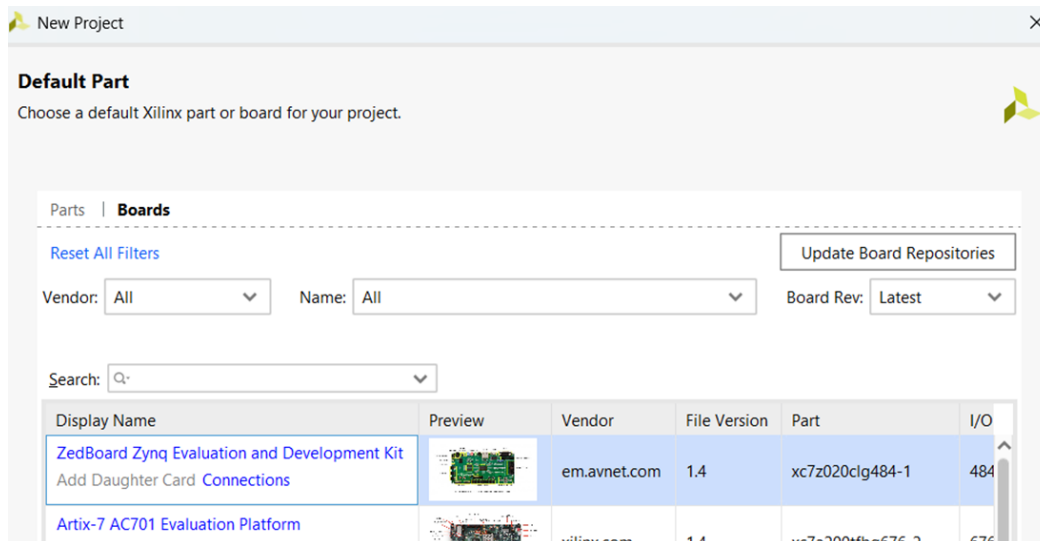
Ở màn hình Add Constraints, có thể chưa thêm file ràng buộc chân I/O vì khi chọn đúng board, Vivado đã có sẵn thông tin về các tài nguyên như LED, nút nhấn, clock và các giao diện ngoại vi.



Hình 4: Add Constraints - bỏ qua bước thêm file ràng buộc ban đầu

Bước 5: Chọn board phát triển

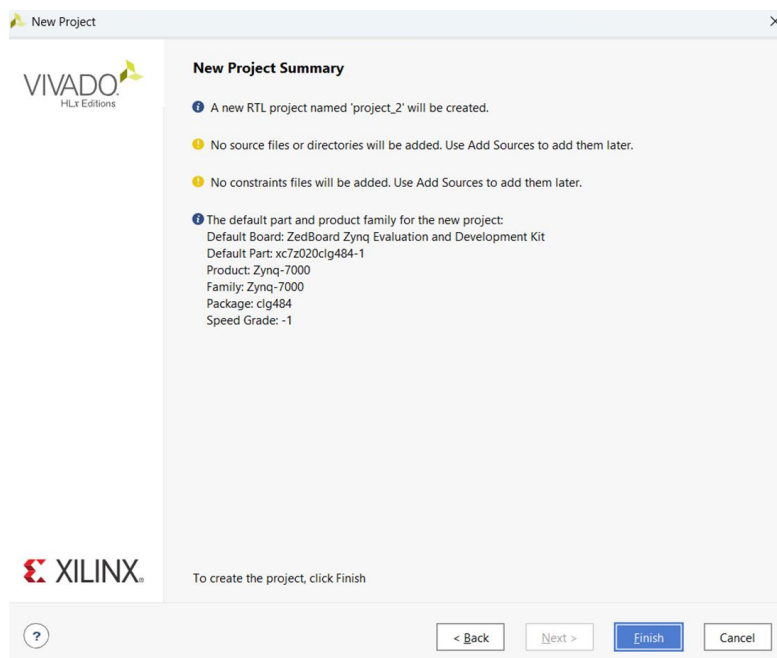
Tại bước chọn phần cứng mục tiêu, tìm và chọn board ZedBoard Zynq Evaluation and Development Kit. Việc chọn đúng board giúp Vivado tự động nhận các preset phần cứng và hỗ trợ kết nối ngoại vi trong các bước sau.



Hình 5: Default Part - chọn board ZedBoard Zynq Evaluation and Development Kit

Bước 6: Xác nhận thông tin Project

Màn hình New Project Summary hiển thị toàn bộ cấu hình đã chọn. Kiểm tra lại tên project, loại project và board mục tiêu, sau đó nhấn Finish để Vivado tạo project.



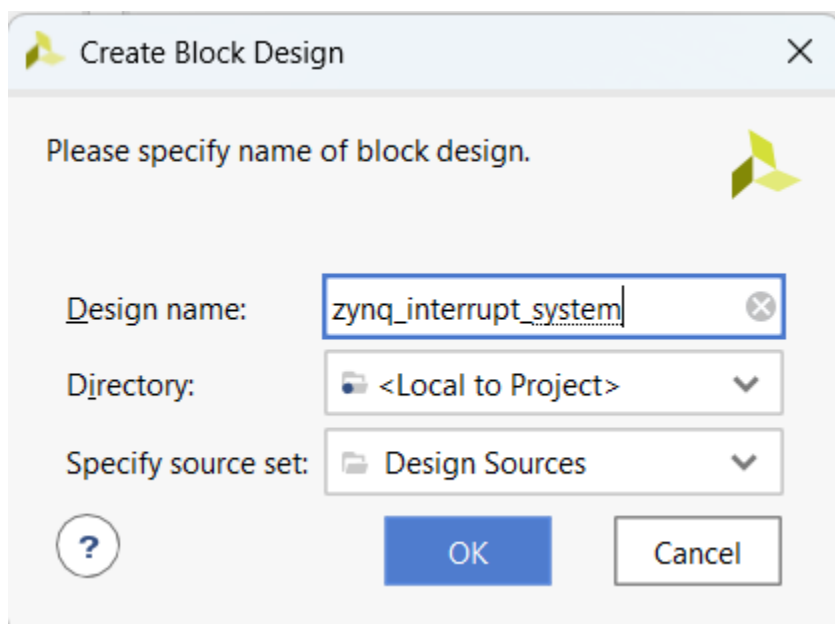
Hình 6: New Project Summary - xác nhận cấu hình trước khi tạo Project

2B. Thiết kế phần cứng bằng Block Design

Phần này xây dựng hệ thống phần cứng gồm ZYNQ7 Processing System, các khối AXI GPIO, kết nối nút nhấn, LED và đường interrupt từ PL về PS.

Bước 7: Tạo Block Design mới

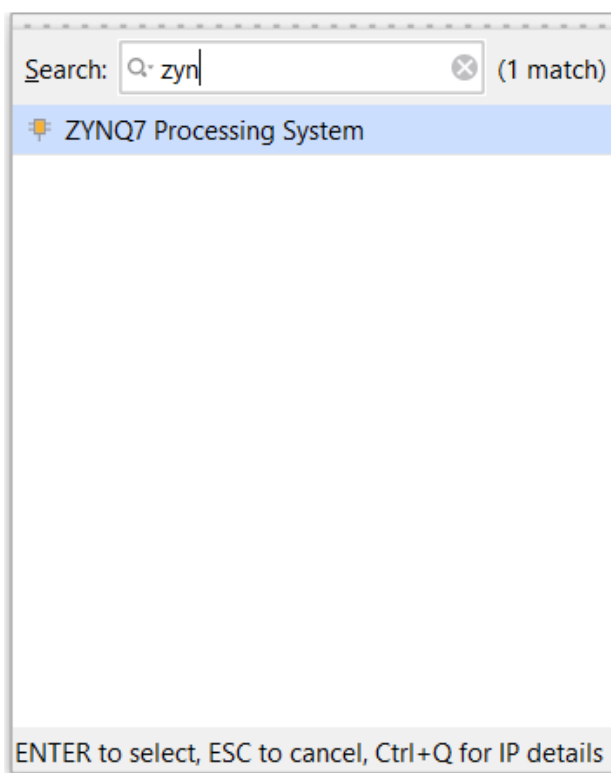
Sau khi tạo project, vào IP Integrator và chọn Create Block Design. Đặt tên block design là zynq_interrupt_system để thể hiện mục tiêu thiết kế hệ thống Zynq có sử dụng interrupt.



Hình 7: Create Block Design - đặt tên zynq_interrupt_system

Bước 8: Thêm ZYNQ7 Processing System

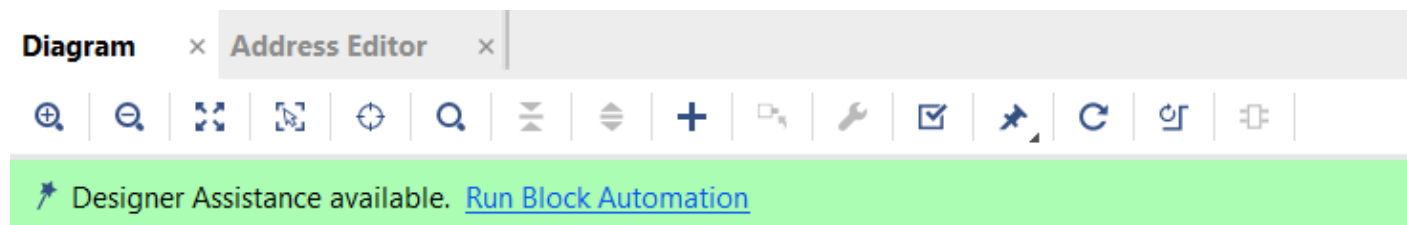
Trong canvas Block Design, dùng Add IP và tìm từ khóa zynq. Chọn ZYNQ7 Processing System để thêm khối Processing System vào thiết kế.



Hình 8: Add IP - tìm ZYNQ7 Processing System

Bước 9: Kiểm tra Processing System trên canvas

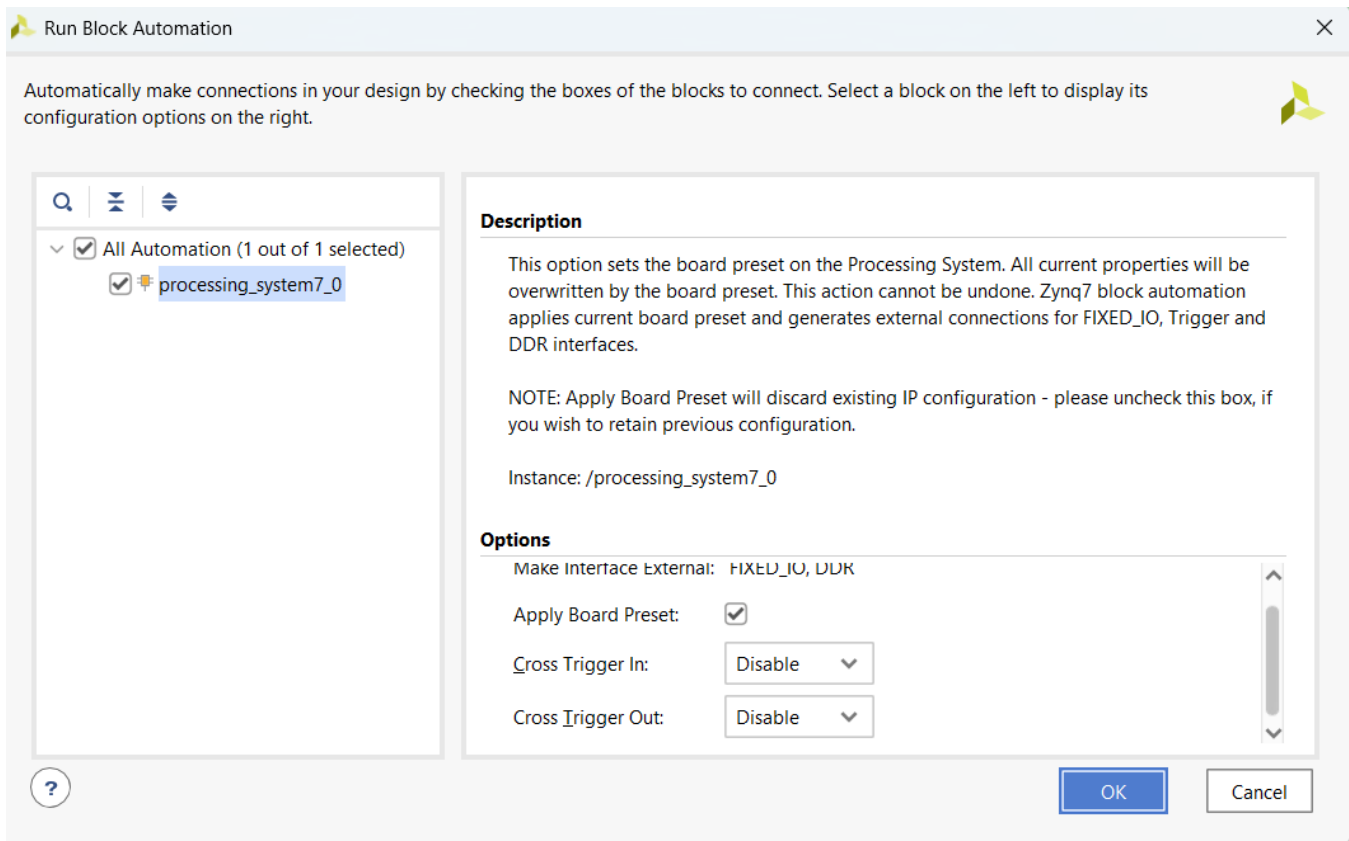
Sau khi thêm ZYNQ7 Processing System, Vivado hiển thị khối PS trên canvas và xuất hiện gợi ý Designer Assistance. Đây là tín hiệu cho biết có thể chạy Block Automation để cấu hình nhanh hệ thống.



Hình 9: Block Design - Processing System đã được thêm vào canvas

Bước 10: Chạy Block Automation

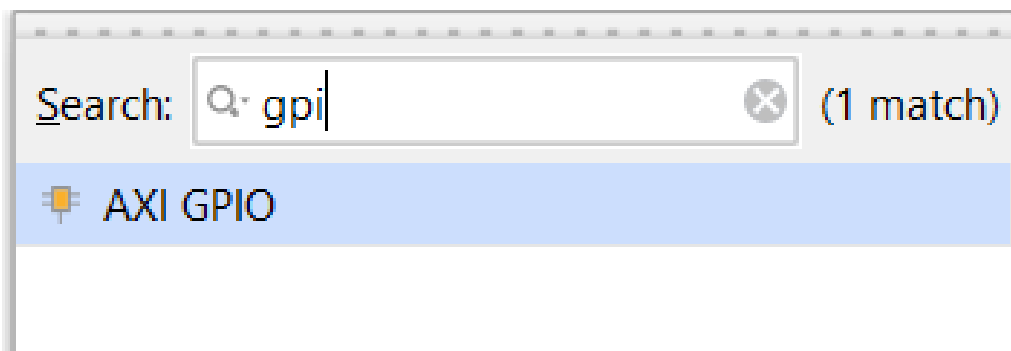
Chọn Run Block Automation để Vivado tự động cấu hình các thành phần cơ bản của Zynq Processing System, bao gồm clock, reset, DDR và các thiết lập liên quan đến board đang sử dụng.



Hình 10: Run Block Automation - cấu hình tự động cho ZYNQ7 Processing System

Bước 11: Thêm AXI GPIO

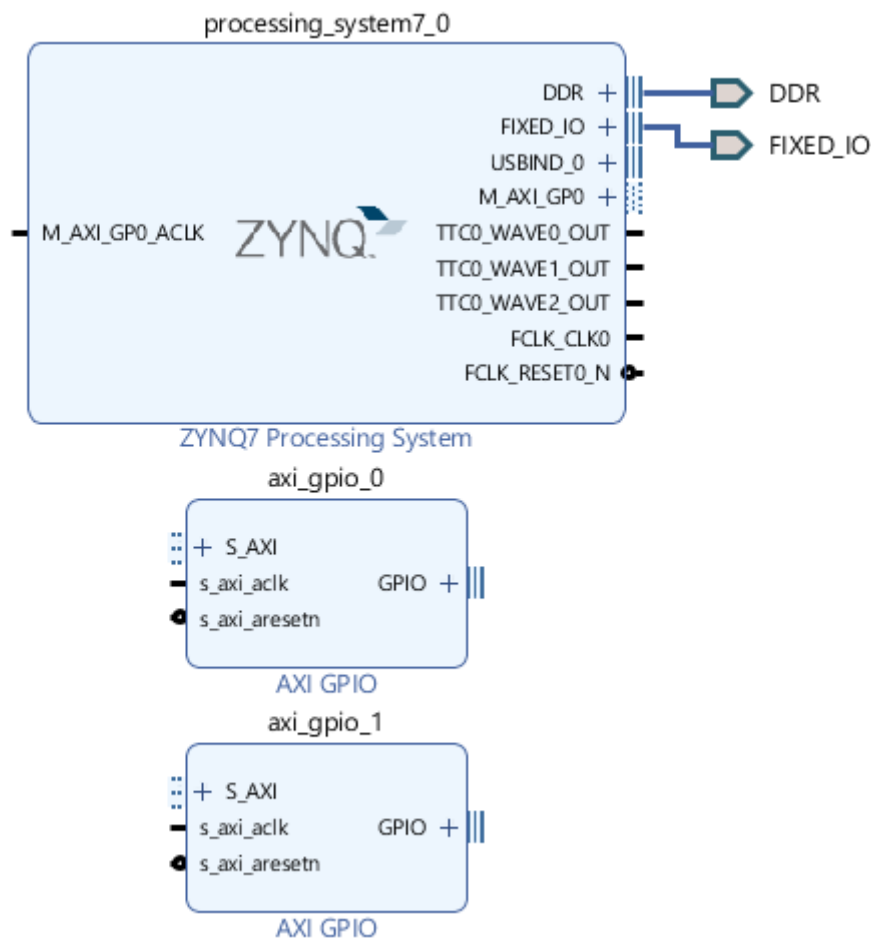
Tiếp tục dùng Add IP và tìm từ khóa gpio để thêm khối AXI GPIO vào Block Design. Trong bài Lab này, AXI GPIO được dùng để kết nối với nút nhấn và LED trên board.



Hình 11: Add IP - tìm và thêm AXI GPIO

Bước 12: Quan sát các IP ban đầu trong Block Design

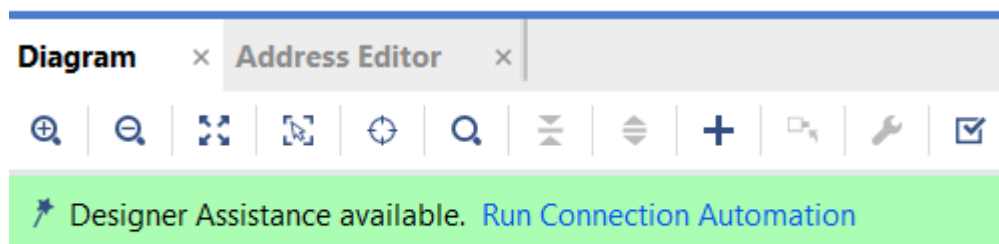
Sau khi thêm các IP cần thiết, canvas hiển thị ZYNQ7 Processing System cùng các khối AXI GPIO. Các khối này vẫn cần được kết nối bus AXI, clock và reset trước khi hệ thống có thể hoạt động.



Hình 12: Block Design - thêm các khối AXI GPIO ban đầu

Bước 13: Chạy Connection Automation

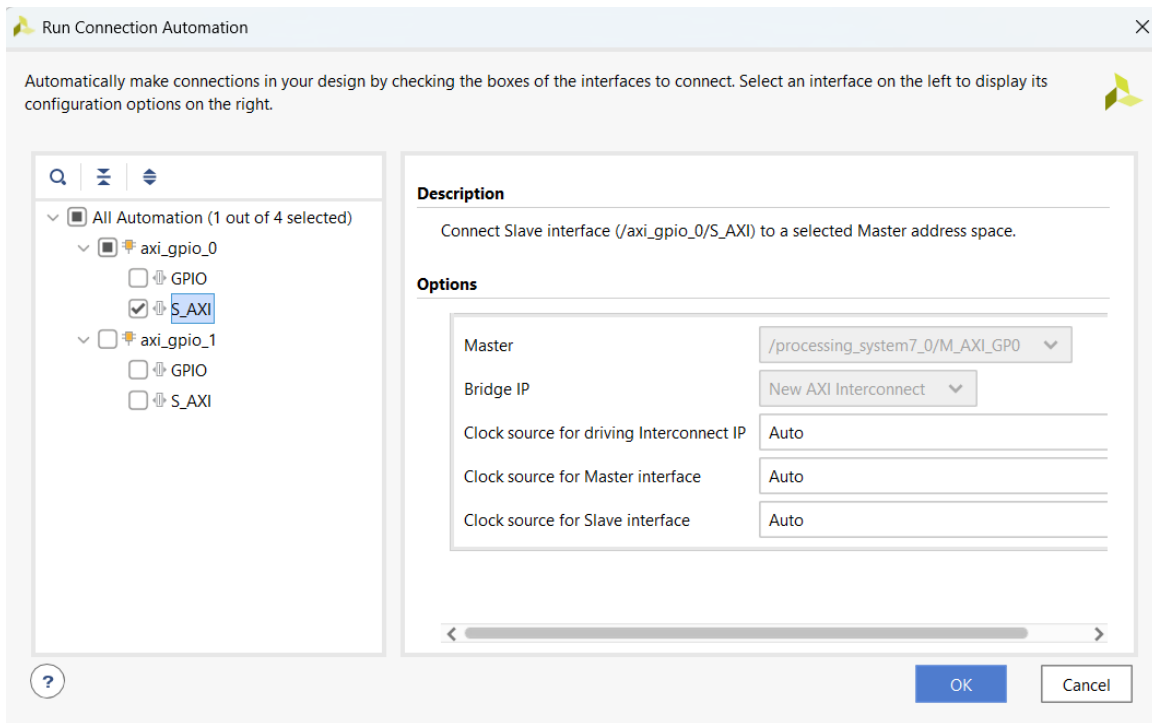
Vivado hiển thị thông báo Run Connection Automation để hỗ trợ nối tự động các giao diện còn thiếu. Chức năng này giúp thêm các khối trung gian như AXI Interconnect và Processor System Reset nếu cần.



Hình 13: Run Connection Automation - tự động nối các giao diện cần thiết

Bước 14: Kết nối giao diện S_AXI của axi_gpio_0

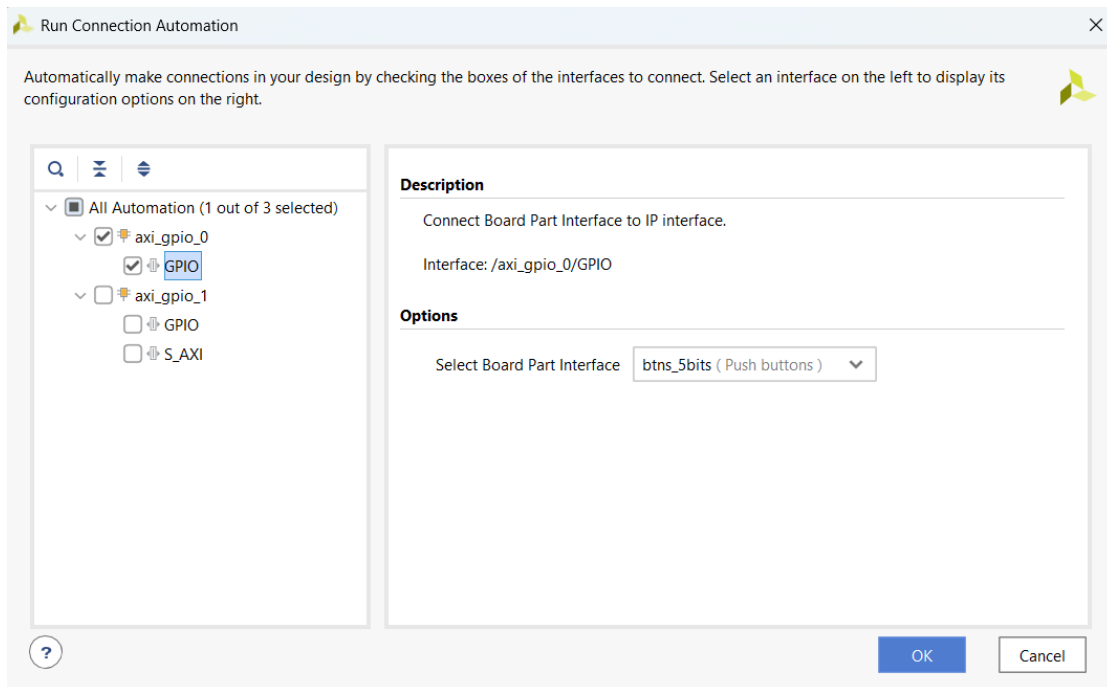
Trong hộp thoại Run Connection Automation, chọn giao diện S_AXI của axi_gpio_0 để kết nối với Master AXI của Processing System. Kết nối này cho phép phần mềm trong PS truy cập thanh ghi của GPIO qua bus AXI.



Hình 14: Connection Automation - nối S_AXI của axi_gpio_0 với Master AXI

Bước 15: Gán axi_gpio_0 với nút nhấn

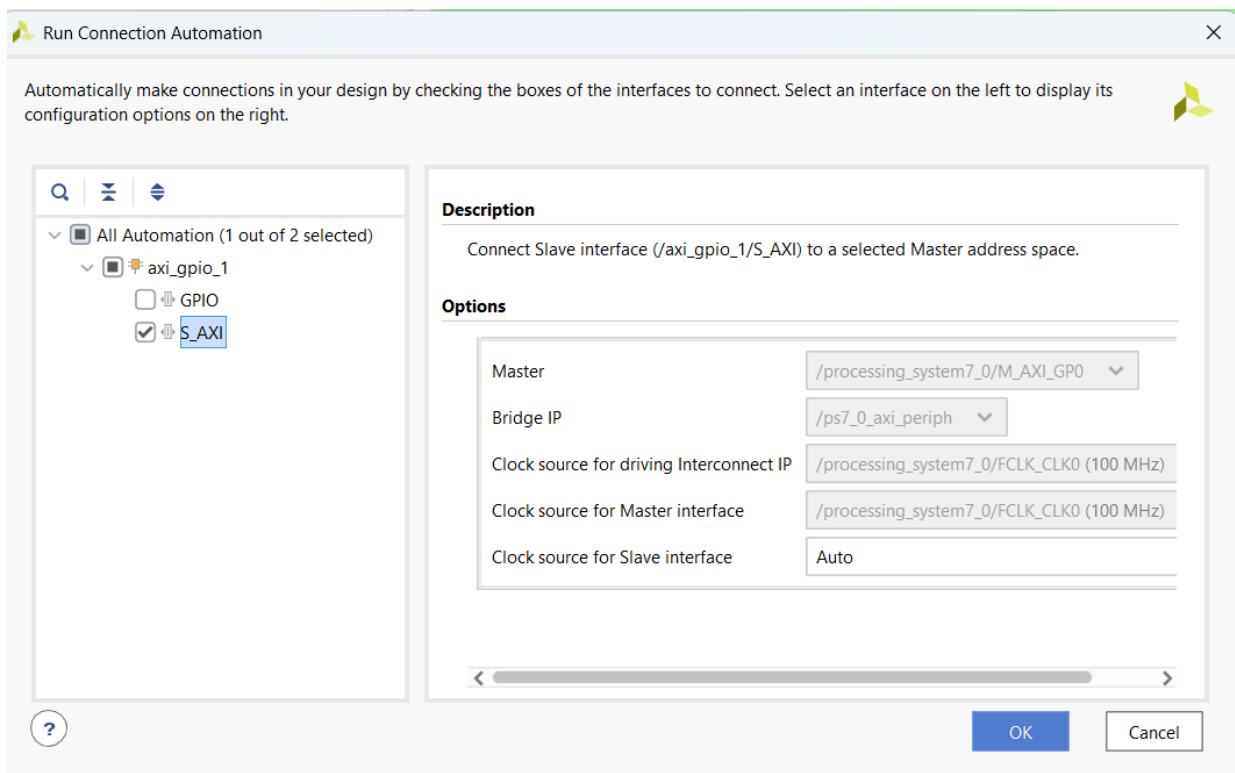
Tại giao diện GPIO của axi_gpio_0, chọn Board Part Interface là btns_5bits. Nhờ đó, khối GPIO này sẽ đọc trạng thái các nút nhấn trên board.



Hình 15: Connection Automation - gán axi_gpio_0/GPIO với btns_5bits

Bước 16: Kết nối giao diện S_AXI của axi_gpio_1

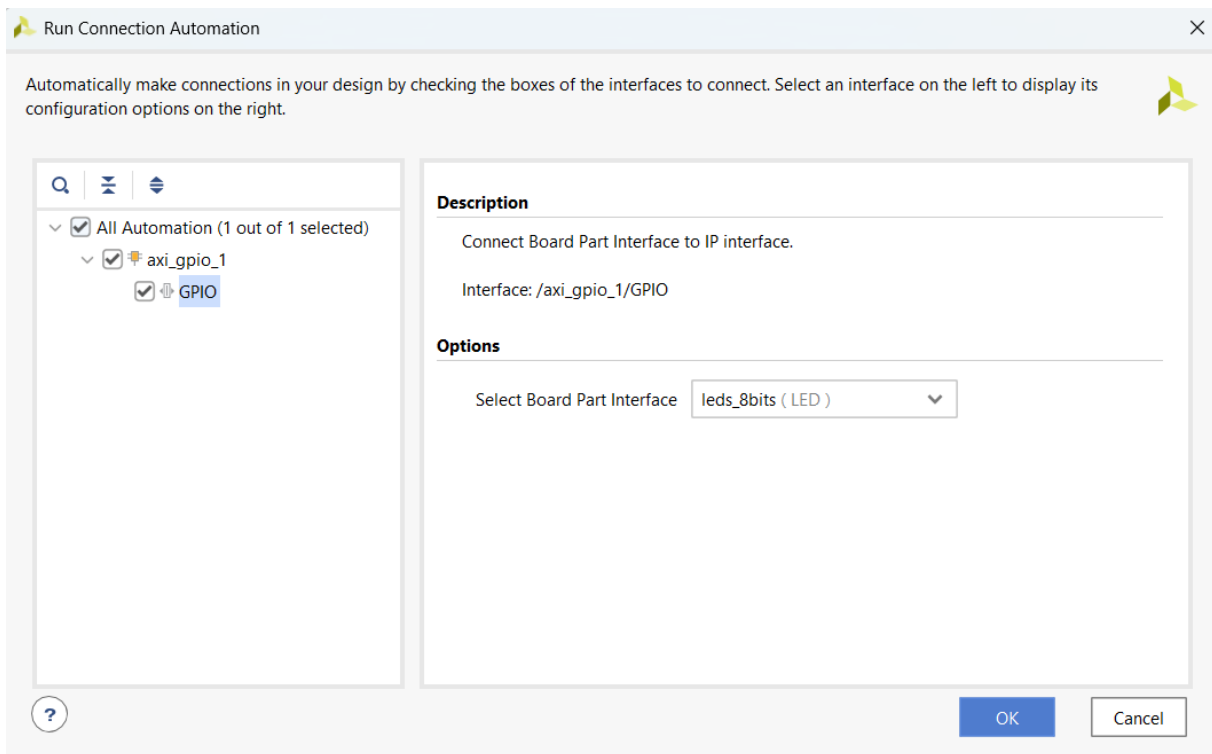
Thực hiện tương tự với axi_gpio_1, kết nối giao diện S_AXI của IP này vào hệ thống AXI. Đây là đường giao tiếp để CPU cấu hình và điều khiển GPIO thứ hai.



Hình 16: Connection Automation - nối S_AXI của axi_gpio_1 với Master AXI

Bước 17: Gán axi_gpio_1 với LED

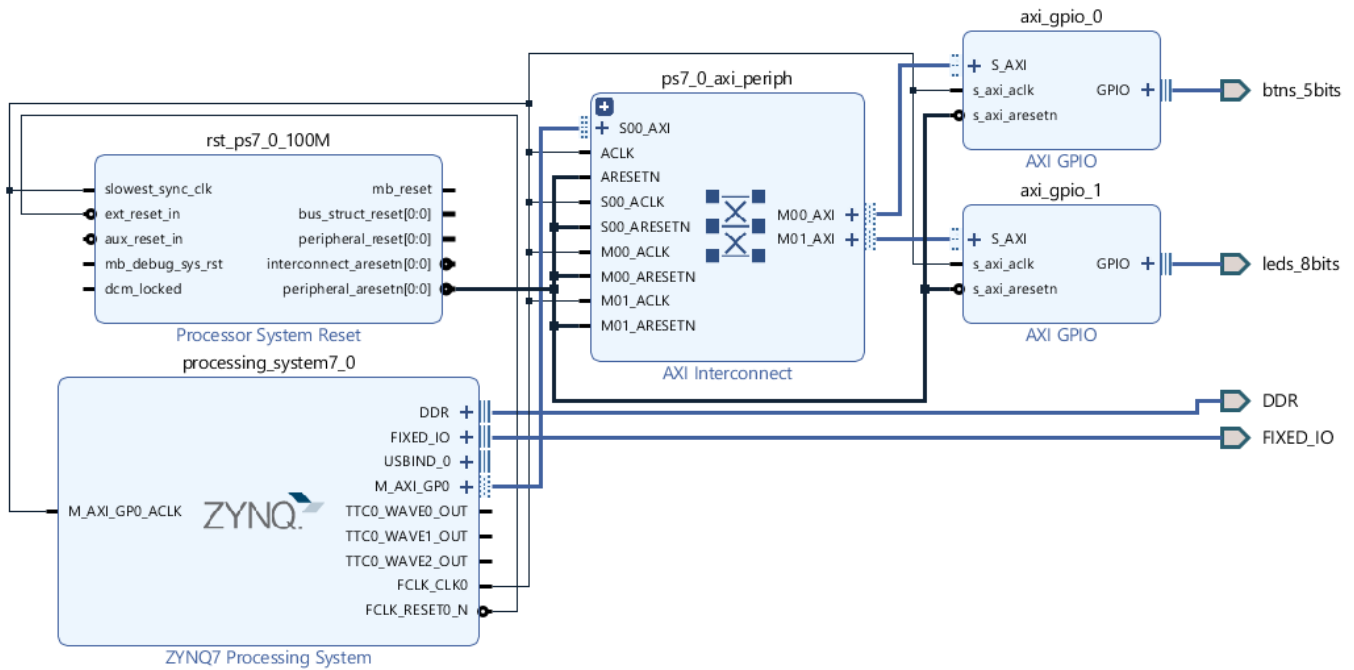
Tại giao diện GPIO của axi_gpio_1, chọn Board Part Interface là leds_8bits. Khối GPIO này sẽ đóng vai trò điều khiển trạng thái các LED trên board.



Hình 17: Connection Automation - gán axi_gpio_1/GPIO với leds_8bits

Bước 18: Kiểm tra sơ đồ sau Connection Automation

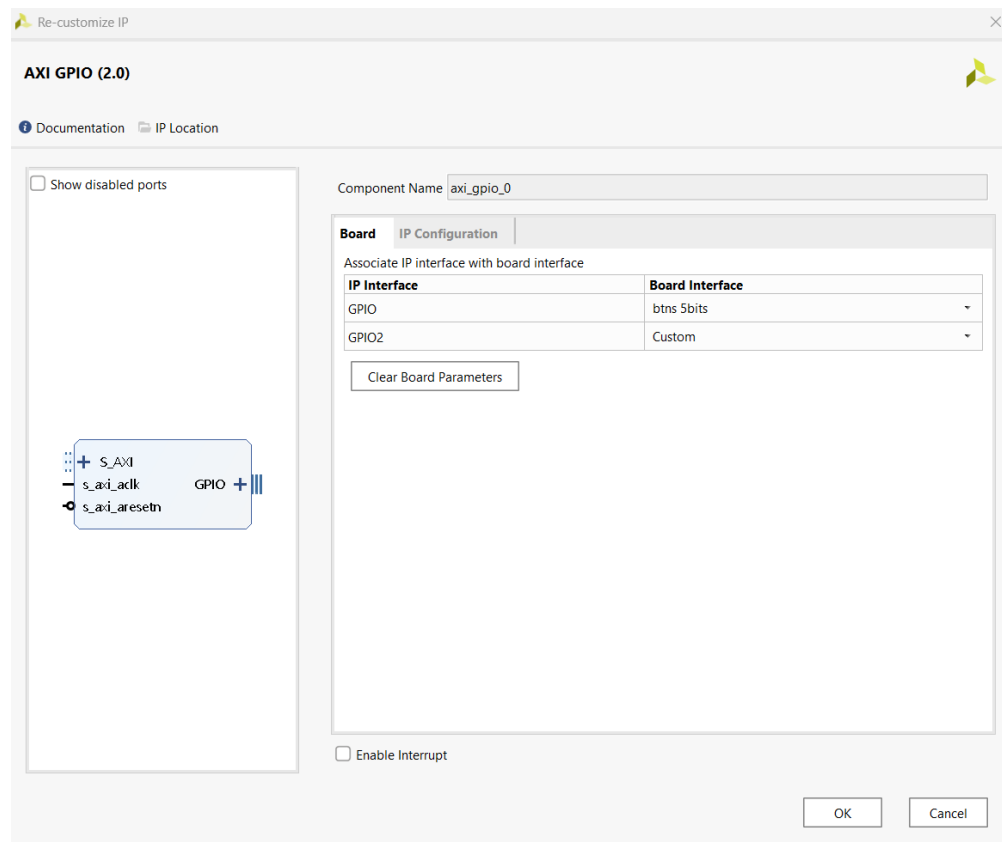
Sau khi Connection Automation hoàn tất, sơ đồ Block Design thể hiện đầy đủ kết nối giữa Zynq Processing System, AXI Interconnect, Processor System Reset, AXI GPIO, nút nhấn và LED.



Hình 18: Block Design - hoàn tất kết nối nút nhấn và LED

Bước 19: Kiểm tra cấu hình board interface của axi_gpio_0

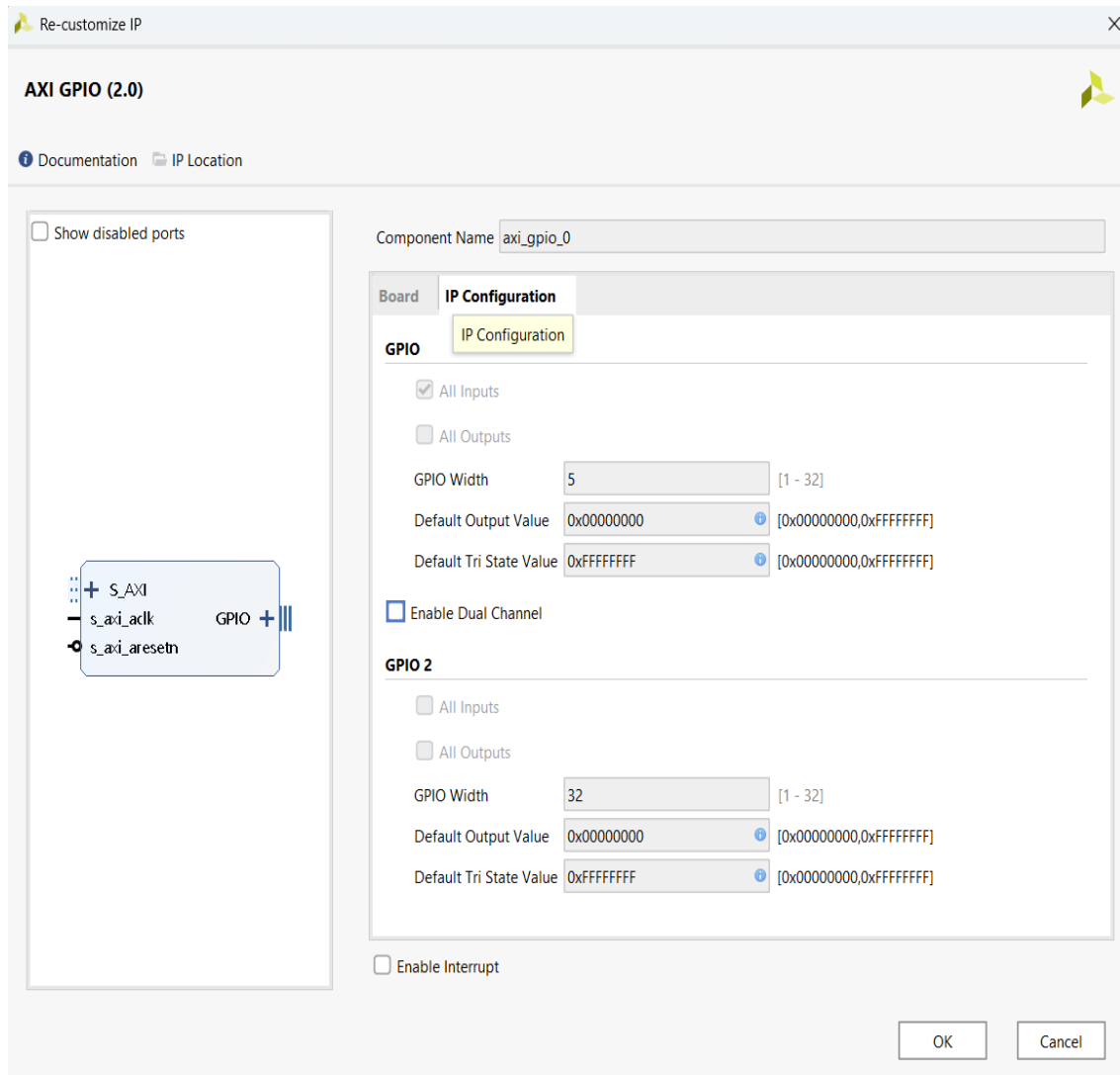
Mở lại cấu hình của axi_gpio_0 để kiểm tra cổng GPIO đã được liên kết với btns_5bits. Việc kiểm tra này giúp đảm bảo nguồn tín hiệu nút nhấn được ánh xạ đúng vào GPIO.



Hình 19: Re-customize IP - kiểm tra board interface của axi_gpio_0

Bước 20: Cấu hình độ rộng GPIO

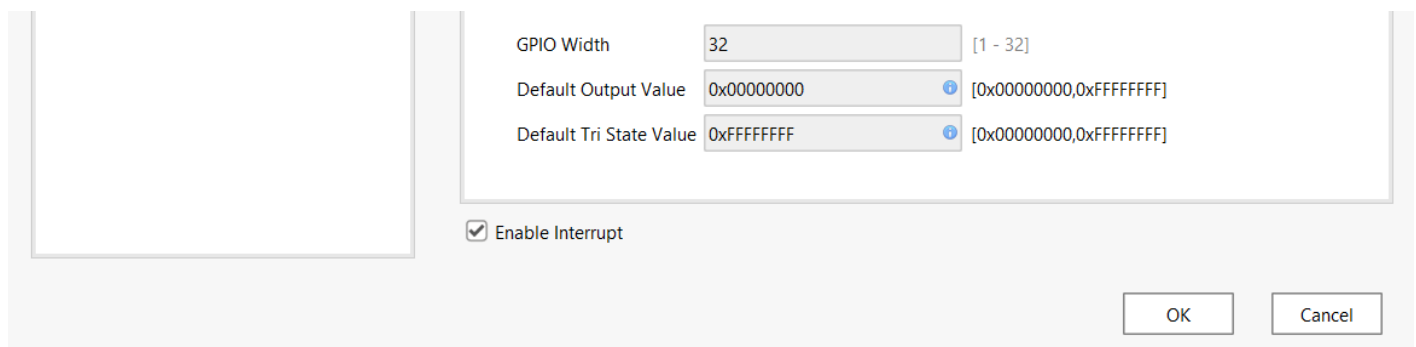
Trong tab IP Configuration, đặt GPIO Width phù hợp với số lượng nút nhấn được sử dụng. Với btns_5bits, độ rộng GPIO cần tương ứng với 5 bit đầu vào.



Hình 20: IP Configuration - đặt độ rộng GPIO cho axi_gpio_0

Bước 21: Bật chức năng interrupt cho AXI GPIO

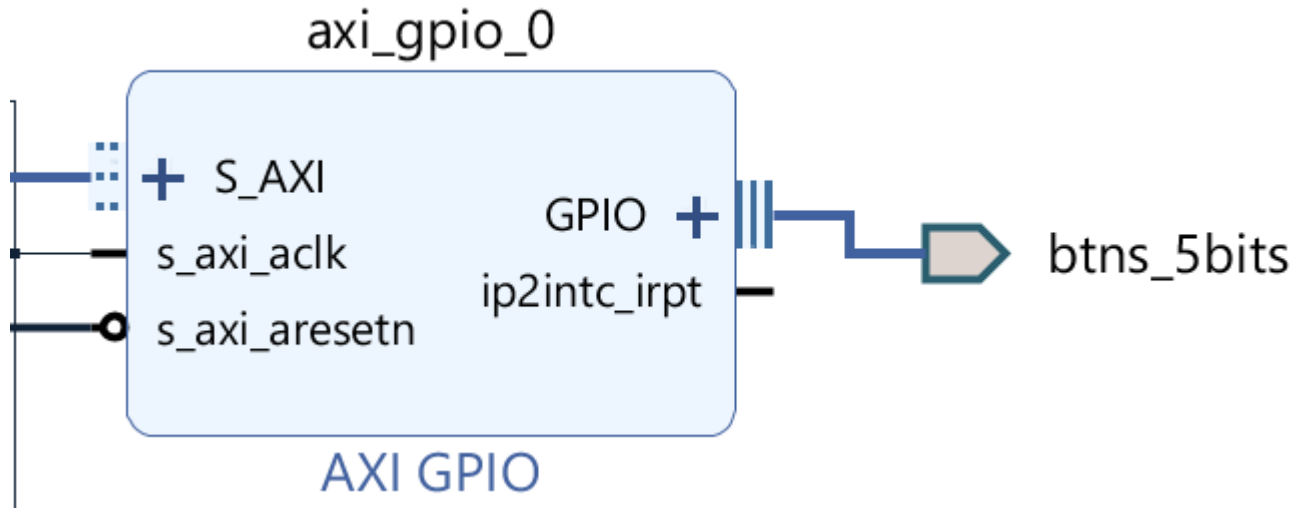
Tích chọn Enable Interrupt để axi_gpio_0 có thể phát tín hiệu ngắt khi xảy ra sự kiện từ nút nhấn. Đây là bước quan trọng để chuyển hệ thống từ polling sang interrupt-driven.



Hình 21: IP Configuration - bật Enable Interrupt cho axi_gpio_0

Bước 22: Kiểm tra cổng ngắt của AXI GPIO

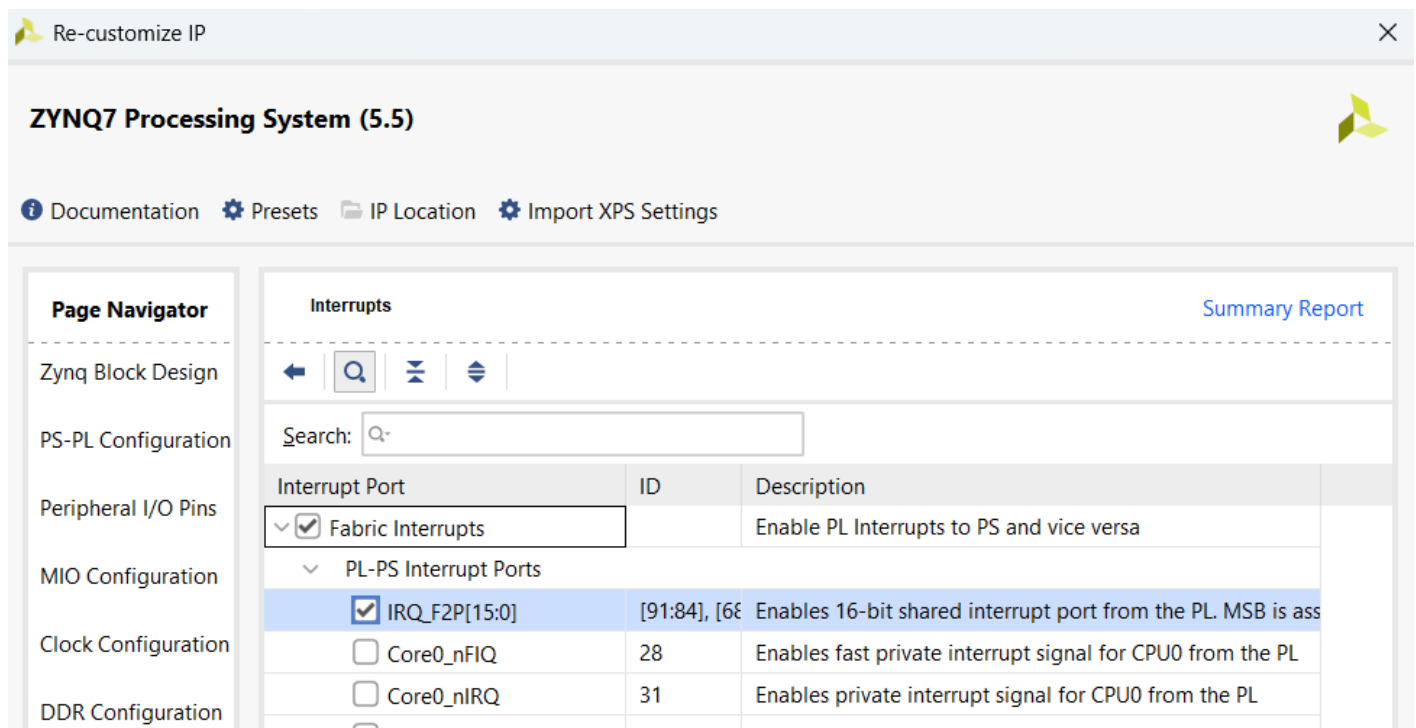
Sau khi bật interrupt, khối AXI GPIO xuất hiện thêm cổng ip2intc_irpt. Cổng này là tín hiệu ngắt từ GPIO và cần được đưa về Processing System.



Hình 22: AXI GPIO - cổng ngắt ip2intc_irpt xuất hiện sau khi bật interrupt

Bước 23: Bật Fabric Interrupt trong ZYNQ7 Processing System

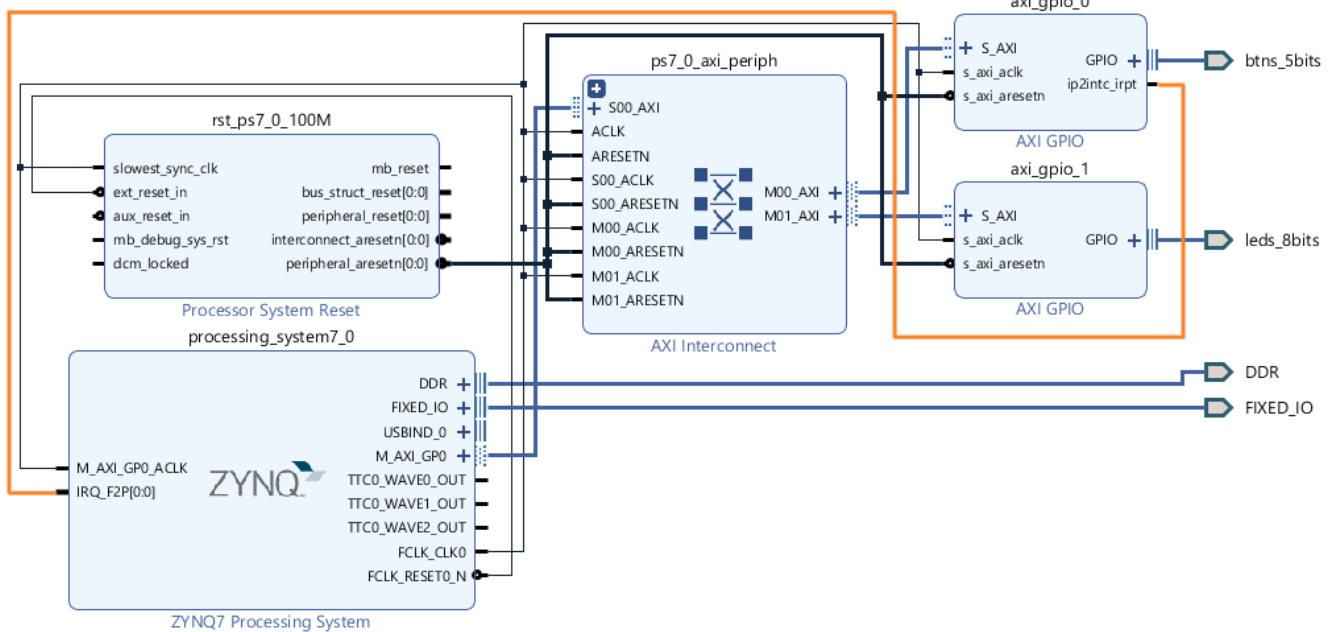
Mở cấu hình ZYNQ7 Processing System, vào phần Interrupts và bật Fabric Interrupts. Khi đó, cổng IRQ_F2P được kích hoạt để PS có thể nhận tín hiệu ngắt từ PL.



Hình 23: ZYNQ7 Processing System - bật Fabric Interrupts và IRQ_F2P

Bước 24: Nối interrupt về Processing System

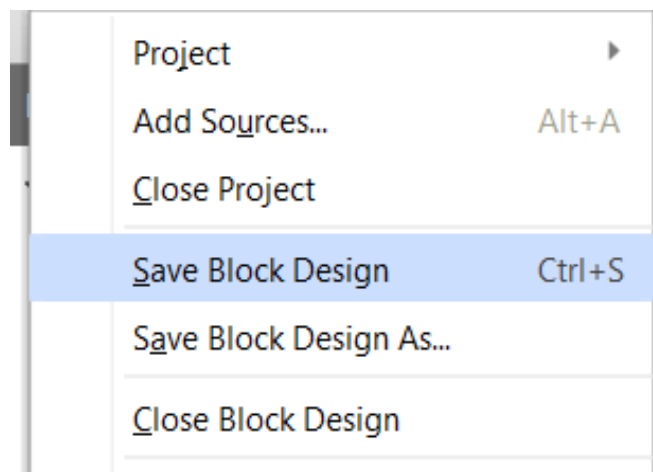
Nối đường ngắt từ AXI GPIO về cổng IRQ_F2P của Processing System. Kết nối này cho phép ARM Cortex-A9 nhận biết sự kiện xảy ra trong phần Programmable Logic.



Hình 24: Block Design - nối đường ngắt từ AXI GPIO về Processing System

Bước 25: Tạo HDL Wrapper

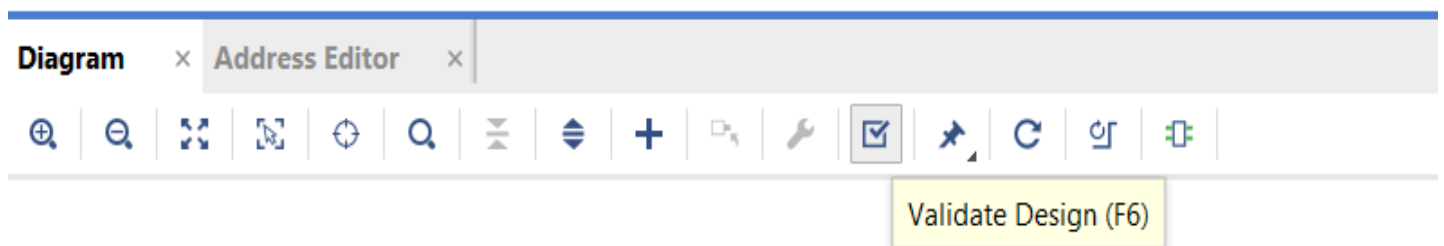
Khi Block Design đã hoàn chỉnh, trong cửa sổ Sources, chuột phải vào file .bd và chọn Create HDL Wrapper. Bước này tạo top-level HDL để Vivado có thể tổng hợp thiết kế.



Hình 25: Sources - tạo HDL Wrapper cho Block Design

Bước 26: Chạy Validate Design

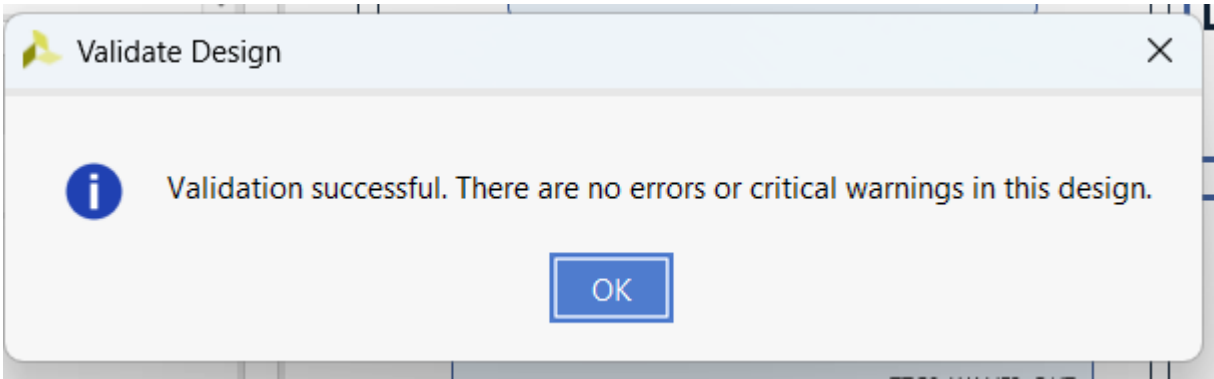
Chọn Validate Design để Vivado kiểm tra các lỗi kết nối, địa chỉ, clock, reset và cấu hình IP. Đây là bước cần thực hiện trước khi Generate Bitstream.



Hình 26: Validate Design - kiểm tra tính hợp lệ của thiết kế

Bước 27: Xác nhận thiết kế hợp lệ

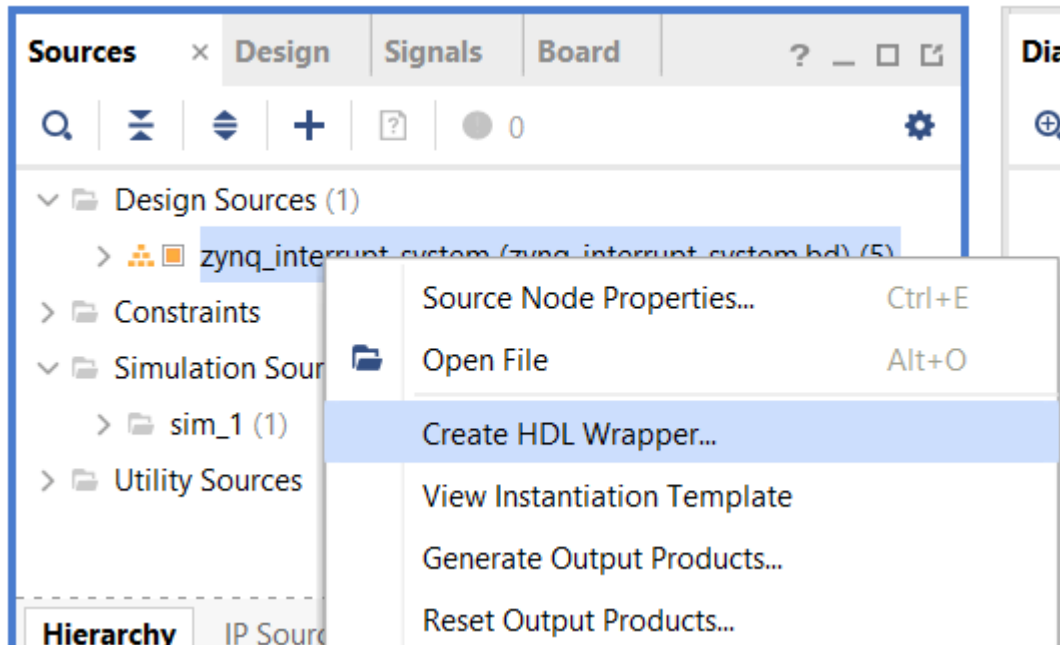
Thông báo Validation Successful cho biết thiết kế không có lỗi nghiêm trọng hoặc cảnh báo quan trọng. Sau bước này có thể tiếp tục quá trình tạo bitstream.



Hình 27: Validation Successful - thiết kế không có lỗi nghiêm trọng

Bước 28: Kiểm tra HDL Wrapper trong Sources

Sau khi tạo HDL Wrapper, kiểm tra lại trong cửa sổ Sources để bảo đảm file wrapper đã được thêm vào project và nằm trong cây nguồn thiết kế.



Hình 28: Sources - HDL Wrapper được tạo cho hệ thống

Bước 29: Chọn Generate Bitstream

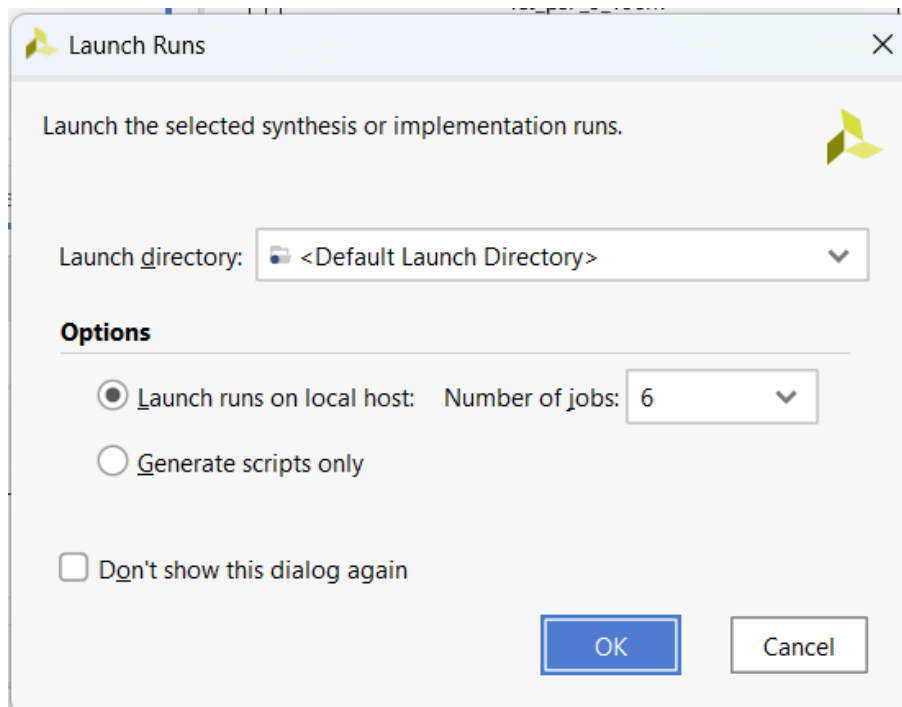
Trong Flow Navigator, chọn Generate Bitstream. Vivado sẽ lần lượt thực hiện Synthesis, Implementation và Bitstream Generation để tạo file cấu hình FPGA.

- ▼ PROGRAM AND DEBUG
 - Generate Bitstream
 - > Open Hardware Manager

Hình 29: Program and Debug - chọn Generate Bitstream

Bước 30: Khởi chạy quá trình tạo bitstream

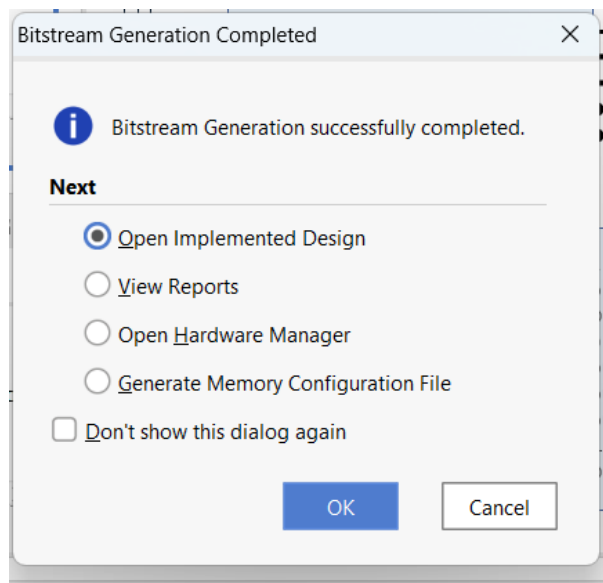
Ở hộp thoại Launch Runs, giữ cấu hình mặc định hoặc chọn số luồng xử lý phù hợp, sau đó xác nhận để Vivado bắt đầu chạy synthesis và implementation.



Hình 30: Launch Runs - khởi chạy Synthesis và Implementation

Bước 31: Hoàn tất Generate Bitstream

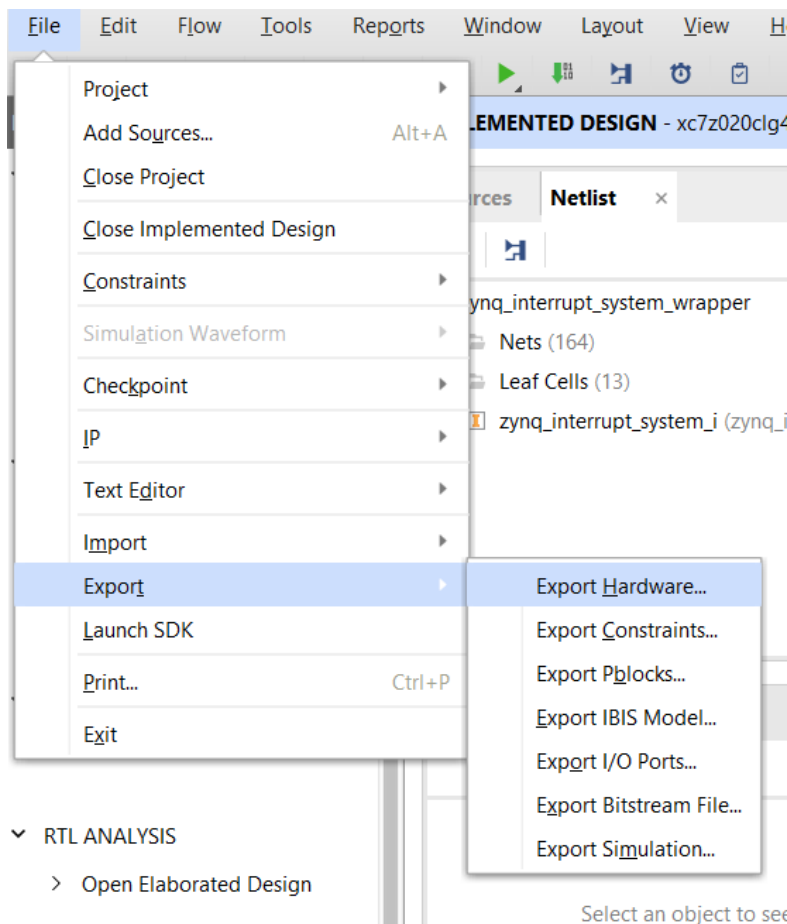
Khi quá trình tạo bitstream hoàn tất, Vivado hiển thị thông báo Bitstream Generation Completed. File .bit lúc này đã sẵn sàng để xuất cùng hardware platform.



Hình 31: Bitstream Generation Completed - tạo Bitstream thành công

Bước 32: Mở chức năng Export Hardware

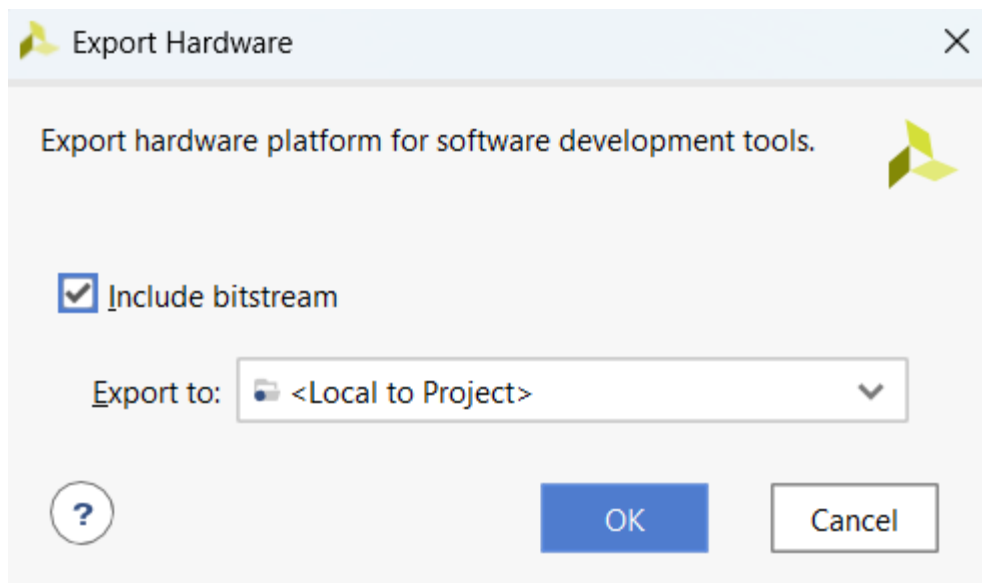
Sau khi tạo bitstream, chọn Export Hardware để xuất thông tin phần cứng sang môi trường SDK hoặc Vitis. File xuất ra chứa mô tả phần cứng, địa chỉ IP và thông tin clock.



Hình 32: Export Hardware - xuất phần cứng sau khi tạo Bitstream

Bước 33: Xuất phần cứng kèm bitstream

Trong hộp thoại Export Hardware, tích chọn Include bitstream. Tùy chọn này giúp hardware platform có kèm file cấu hình FPGA để phục vụ bước nạp mạch sau này.



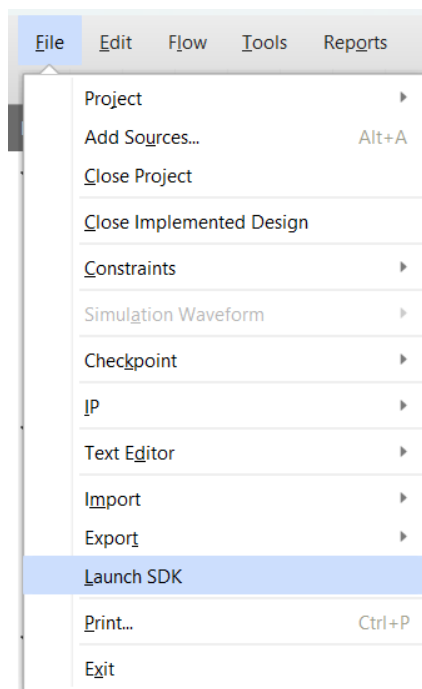
Hình 33: Export Hardware - tích chọn Include Bitstream

2C. Xuất phần cứng và lập trình phần mềm

Phần này xuất hardware platform sang SDK, tạo Application Project, import mã nguồn C và nạp bitstream xuống FPGA.

Bước 34: Khởi chạy Xilinx SDK

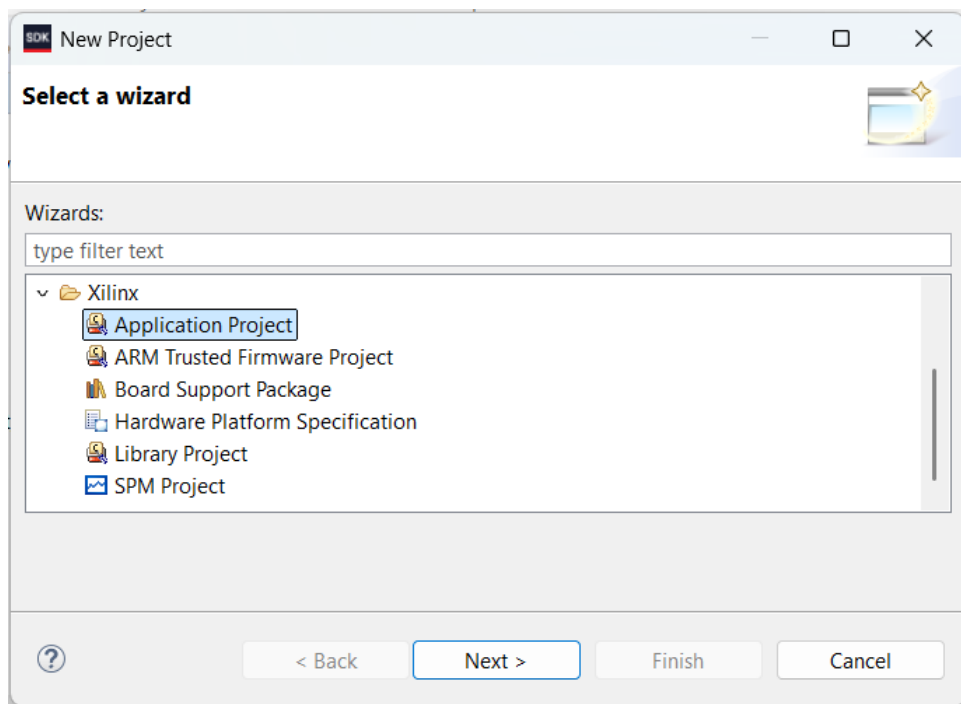
Sau khi export hardware, chọn Launch SDK để mở môi trường phát triển phần mềm. SDK sẽ sử dụng hardware platform vừa xuất để tạo project chạy trên ARM Cortex-A9.



Hình 34: Vivado - Launch SDK để phát triển phần mềm

Bước 35: Tạo Application Project

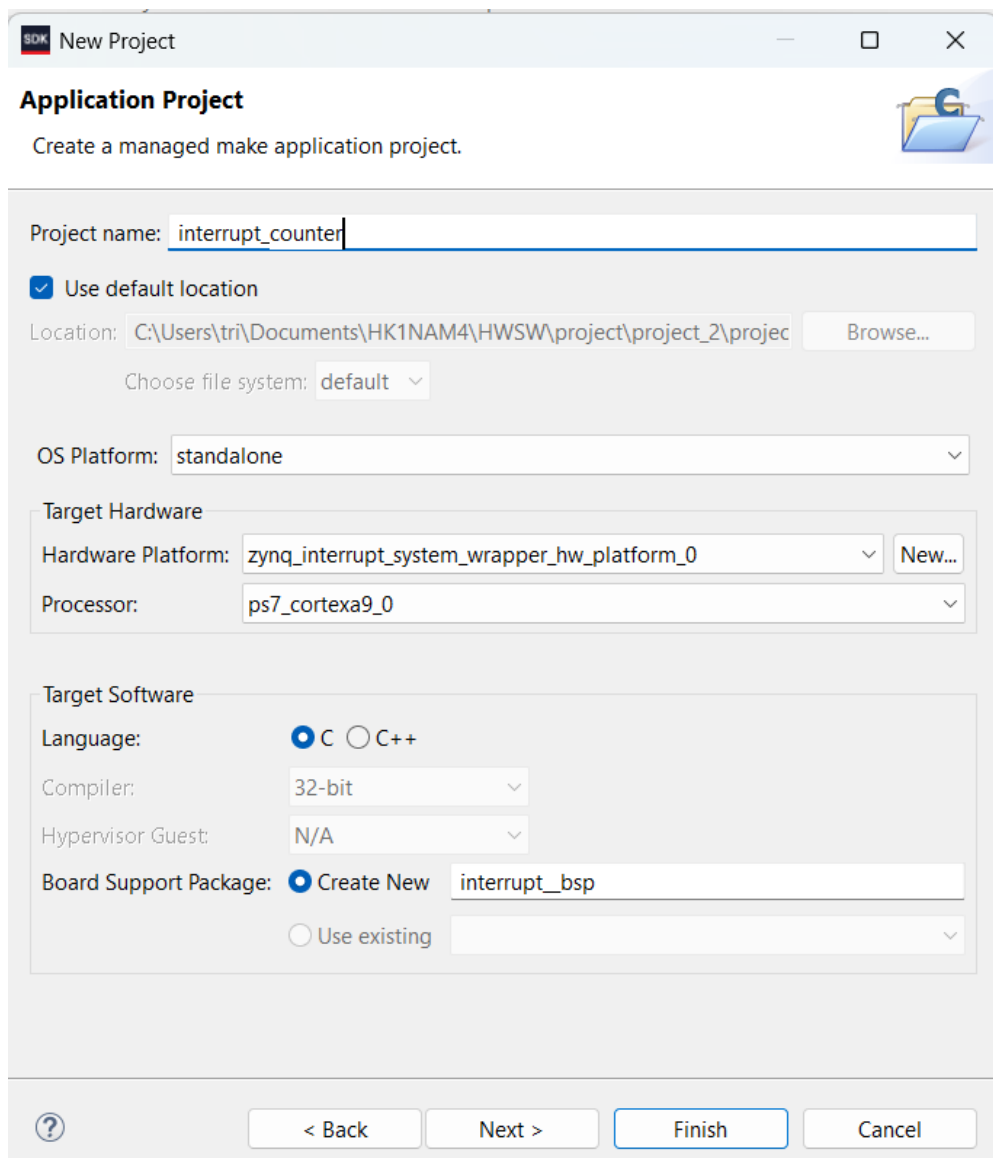
Trong Xilinx SDK, chọn File -> New -> Application Project để tạo chương trình phần mềm chạy trên Processing System. Đây là project chứa mã C xử lý logic điều khiển và interrupt.



Hình 35: Xilinx SDK - tạo Application Project mới

Bước 36: Cấu hình Application Project

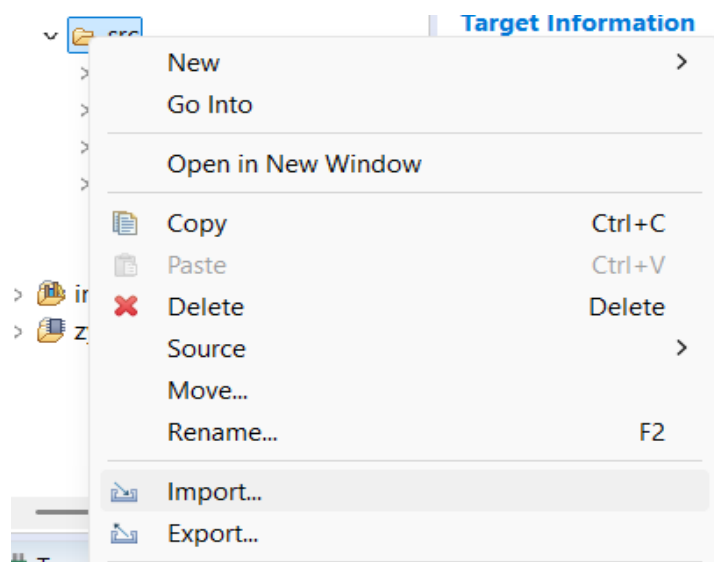
Đặt tên project là `interrupt_counter`, chọn đúng Hardware Platform và processor `ps7_cortexa9_0`. SDK đồng thời tạo BSP để cung cấp driver cho các IP trong hệ thống.



Hình 36: Application Project - đặt tên `interrupt_counter` và chọn Hardware Platform

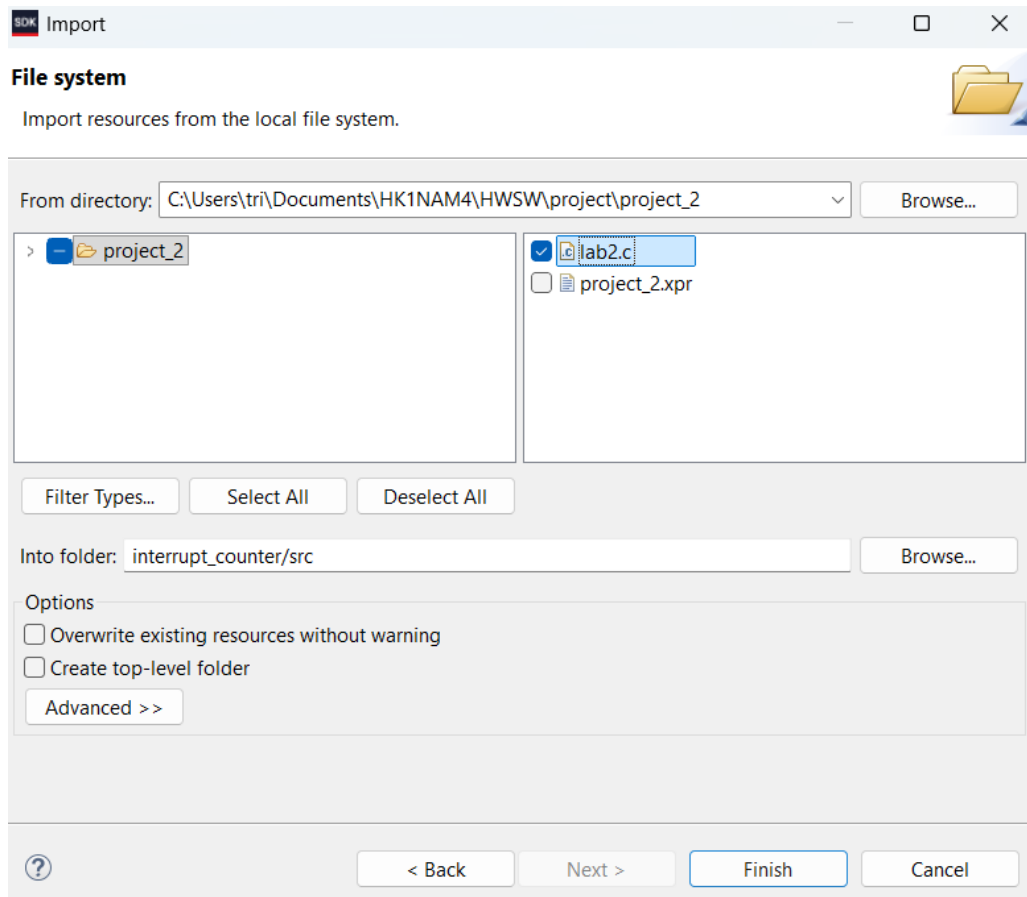
Bước 37: Mở chức năng Import mã nguồn

Trong SDK, dùng chức năng Import để đưa file mã nguồn C đã chuẩn bị vào project. Đây là bước đưa chương trình xử lý interrupt vào thư mục src.



Bước 38: Chọn file mã nguồn C

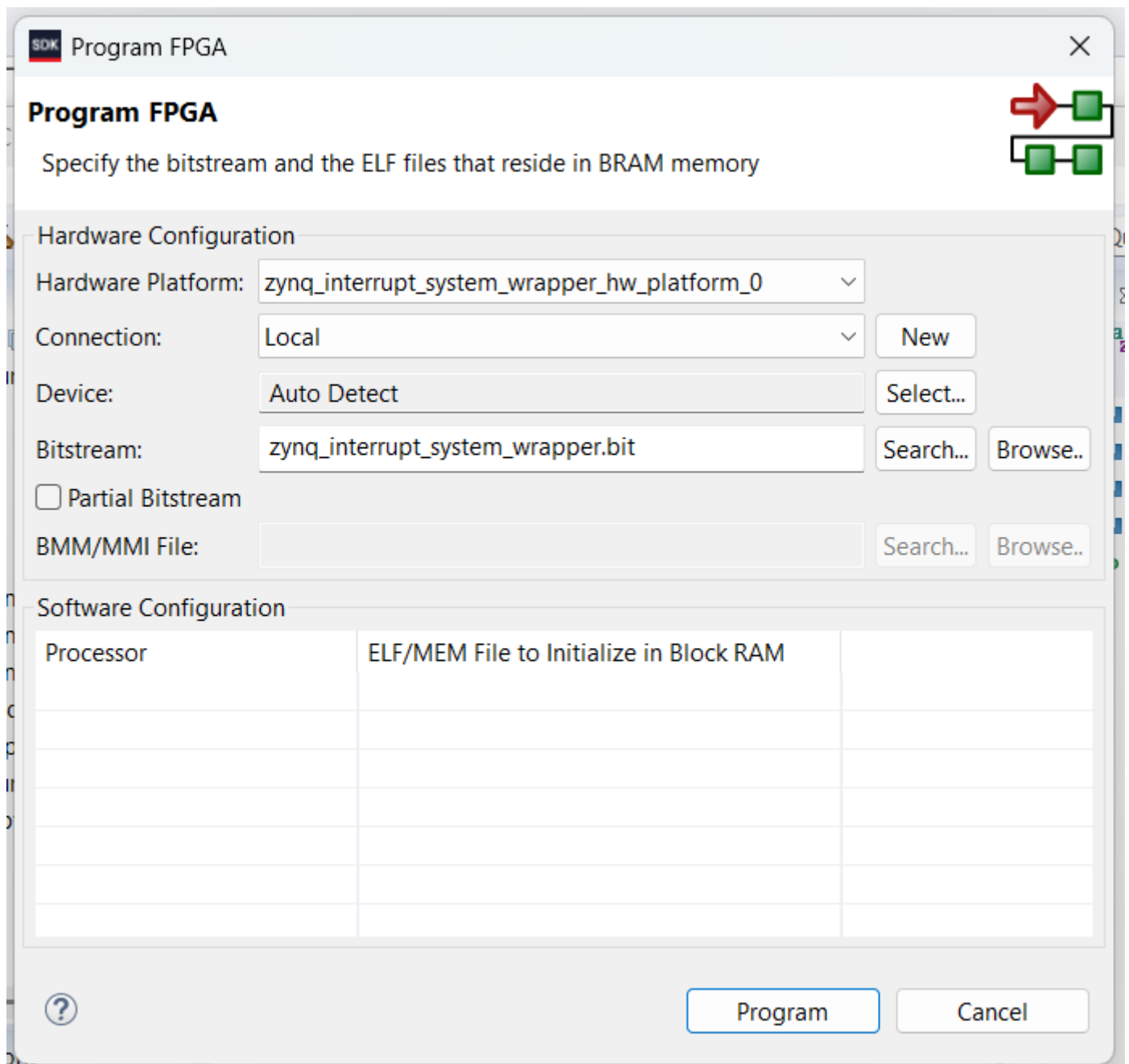
Trong hộp thoại Import File System, chọn file lab2.c và thư mục đích interrupt_counter/src. Sau khi import, SDK sẽ biên dịch mã nguồn cùng project ứng dụng.



Hình 38: Import File System - chọn lab2.c và thư mục interrupt_counter/src

Bước 39: Nạp bitstream xuống FPGA

Cuối cùng, mở Program FPGA, chọn đúng hardware platform và bitstream đã tạo. Nhấn Program để nạp cấu hình phần cứng xuống vùng PL của chip Zynq.



Hình 39: Program FPGA - nạp Bitstream xuống bo mạch

Link github tham khảo: https://github.com/tranntnd27-lab/HW_SW_Projects

Hình nhóm:

